

Session-Aware Evaluation of Foundation-Model Features for Four-Class Acute Lymphoblastic Leukemia Cell Classification: Shortcut Auditing and Leakage Analysis on a Public Peripheral Blood Smear Dataset

AUTHOR NAME(S) - FILL IN
DEPARTMENT, INSTITUTION, CITY, COUNTRY - FILL IN
Correspondence: EMAIL - FILL IN

ABSTRACT

Acute lymphoblastic leukemia (ALL) is diagnosed in part from the appearance of lymphoblasts on peripheral blood smears, and many studies routinely report very high classification accuracy on public smear datasets. These datasets rarely contain patient identifiers, so it is unclear how much of the reported accuracy reflects morphology and how much reflects the way the images were acquired. I analysed a public dataset of 3256 smear images in four classes (Benign, Early, Pre and Pro; 36167 segmented cells) with a leakage-aware protocol. Images were stain-normalised, denoised and segmented, and each cell was isolated with the background replaced by a neutral fill. Capture order was used to build 90 session groups and contiguous cross-validation folds with an embargo. A frozen hematology foundation model (DinoBloom-S) with an attention-pooling head reached a balanced accuracy of 0.968 (95% CI 0.955-0.978) and a macro-F1 of 0.967. Background colour alone, however, predicted the class with a balanced accuracy of 0.823, and keeping the background raised the accuracy of the same model to 0.978. Random block folds overestimated accuracy relative to contiguous folds, and removing training images that were close in capture number to the test images lowered accuracy more than removing the same number at random. Because each class was imaged in separate sessions, in-dataset accuracy is an upper bound on performance on new patients. I conclude that reporting session-

aware folds, shortcut audits and external validation should be a minimum standard for leukemia image classification, and I release the code and every result file used in this paper.

Keywords: acute lymphoblastic leukemia; peripheral blood smear; foundation model; DinoBloom; data leakage; shortcut learning

INTRODUCTION

Acute lymphoblastic leukemia (ALL) is the most common cancer of childhood and is characterised by the uncontrolled proliferation of immature lymphoid cells in the bone marrow and blood. Examination of a stained peripheral blood smear (PBS) is an inexpensive first step in diagnosis, but it depends on the availability of trained hematologists, and agreement between observers on the morphology of individual blasts is imperfect. Automated analysis of PBS images has therefore been studied for more than a decade, including classical image-processing pipelines (Labati et al. 2011) and, more recently, deep convolutional networks and vision transformers (Ghaderzadeh et al. 2022; Matek et al. 2019).

Public datasets have made this research possible, and the reported accuracies are very high. High accuracy on a small public dataset is, however, only informative if the evaluation separates the images used to train a model from those used to test it in the same way that deployment would. Two properties of the data make this difficult. First, public smear datasets rarely publish patient or slide identifiers, so a random split of images puts different images of the same slide into both the training and the test set. Second, if the classes were imaged on different days or with different equipment, a classifier can learn the imaging conditions instead of the cells. The general problem of shortcut learning (Geirhos et al. 2020) has led to spurious predictors being found in radiographic classifiers (DeGrave et al. 2021) and has been described as a leading cause of leakage-driven overoptimism in machine-learning-based science (Kapoor

and Narayanan 2023). A recent benchmark for leukemia classification reports the same concern (Albzour 2026).

The dataset analysed here was released by Aria et al. (2021) and described by Ghaderzadeh et al. (2022). It contains 3256 images of peripheral blood smears in four classes: benign hematogones and three stages of malignant lymphoblasts (Early, Pre and Pro). The dataset documentation reports 89 patients, but the image files carry no patient identifier. I show below that the file numbering, however, preserves the order of image capture, and that this order is informative about both the class and the appearance of neighbouring images.

Recent foundation models offer a way to obtain strong features without large labelled datasets. DINOv2 (Oquab et al. 2024) is a self-supervised vision transformer (Dosovitskiy et al. 2021); DinoBloom (Koch et al. 2024) continues its training on a large collection of single-cell images from hematology, and its weights are openly available. A linear classifier on frozen features of such a model is a strong, cheap and well-defined baseline, and is the model used for the headline result in this paper.

Clinical background

Acute lymphoblastic leukemia is a malignancy of lymphoid progenitor cells (lymphoblasts) that accumulate in the bone marrow, suppress normal blood formation and enter the circulation. It is the most common cancer of children (Hunger and Mullighan 2015). Current classification systems define the disease by the lineage of the malignant cell (B-cell or T-cell precursor) and by genetic subtype (Arber et al. 2016). Treatment depends strongly on the subtype and on the response to the first phase of therapy, so rapid and accurate recognition of the disease at presentation matters for the patient.

The diagnostic work-up combines several investigations. A complete blood count indicates abnormal numbers of blood cells. The peripheral blood smear, a drop of blood spread on a glass slide and stained with a Romanowsky-type stain, allows a hematologist to inspect the morphology of individual white blood cells under the microscope. Confirmation and subtyping rely on bone-marrow examination, flow cytometry for cell-surface markers, and cytogenetic and molecular tests. The smear is thus a screening step, not the final diagnosis, and image-based tools for the smear aim to support that screening step, for example by flagging smears that contain suspicious cells.

In a smear, lymphoblasts differ from mature lymphocytes and from normal immature cells in size, in the ratio of nucleus to cytoplasm, in the texture of the chromatin and in the presence of nucleoli. These differences are graded and overlap between classes, which is one reason why agreement between observers is imperfect and why an automated and reproducible description of cell appearance is attractive.

The dataset and its four classes

The dataset used in this study (Aria et al. 2021; Ghaderzadeh et al. 2022) groups smear images into four classes. The Benign class contains images of hematogones, which are normal immature B-lymphocyte precursors that resemble lymphoblasts and are a classical source of confusion. The three malignant classes correspond to stages of precursor B-cell lymphoblasts, called Early (early pre-B), Pre (pre-B) and Pro (pro-B) in the dataset documentation. The images were acquired from patients and labelled by experts, according to the dataset description, using flow cytometry as the reference standard. I rely on that documentation and could not verify the labelling process independently.

Two practical properties of the dataset shape the analysis in this paper. First, the images are small (224 x 224 pixels) and each contains a handful of cells (on average about 11 per image), so the context available to a model is limited. Second, the dataset provides no table linking the images to patients, slides or acquisition sessions. Both properties are common among public datasets, and the second is the reason why evaluation design receives so much attention here.

Computer-aided analysis of blood smears

Early systems for leukemia detection used the classical pipeline of image analysis: colour-space conversion, thresholding or clustering to find white blood cells, morphological operations and watershed transforms to separate touching cells (Meyer and Beucher 1990), and hand-designed descriptors of shape, colour and texture that were fed to a conventional classifier (Labati et al. 2011). The Haralick texture features (Haralick et al. 1973) and measures such as the nucleus-to-cytoplasm ratio were typical. The strength of this approach is interpretability; its weakness is the dependence of every step on thresholds that must be tuned to a particular staining and imaging set-up.

Deep convolutional networks (Krizhevsky et al. 2012; He et al. 2016) replaced hand-designed descriptors by features learned from labelled images, and efficient architectures (Tan and Le 2019) reduced the cost of training. Encoder-decoder networks (Ronneberger et al. 2015) and generalist cell segmentation tools (Stringer et al. 2021; Schmidt et al. 2018) addressed the segmentation step. Vision transformers (Vaswani et al. 2017; Dosovitskiy et al. 2021) added global self-attention over image patches. For blood smears, large labelled collections such as the single-cell dataset of Matek et al. (2019) made it possible to train and test networks for the recognition of cell types and blasts.

Because labelled medical images are scarce, transfer learning is central: a network trained on a large source dataset is adapted to the target task (Pan and Yang 2010). How well features transfer depends on the architecture and the source data (Kornblith et al. 2019). Self-supervised pre-training (Caron et al. 2021; Oquab et al. 2024) produces general features without labels, and domain-specific pre-training on hematology images (Koch et al. 2024) aims to capture the appearance of blood cells in particular. The simplest and most reproducible way to use such a model is to freeze it, compute an embedding for each image or cell once, and train only a small classifier on top.

Stain variation and normalisation

Smears are stained by hand or by automatic stainers, and the resulting colour depends on the stain batch, the staining time, the thickness of the smear and the light source and camera. Differences in colour between laboratories can degrade the accuracy of classifiers that were trained elsewhere. Colour normalisation methods map the colour distribution of an image to that of a reference. The method of Reinhard et al. (2001) matches the mean and standard deviation of each channel of a decorrelated colour space; methods based on stain separation (Macenko et al. 2009; Vahadane et al. 2016) estimate the stain vectors of each image. Augmentation that perturbs stain colour is an alternative or a complement, and Tellez et al. (2019) reported that colour augmentation can matter more than normalisation for the robustness of classifiers in computational pathology.

Validity of evaluation in medical machine learning

A growing literature documents that reported performance in medical machine learning is often inflated by weaknesses of study design rather than by the models. Varoquaux and Cheplygina (2022) reviewed such failures in medical imaging, including small and biased

datasets and evaluation on data that overlap with the training data. Roberts et al. (2021) found that most machine-learning studies of COVID-19 imaging were at high risk of bias and had methodological flaws that prevented clinical use. Kapoor and Narayanan (2023) described leakage as a cause of failures of reproducibility across scientific fields and gave a taxonomy of leakage types, including the case in which the test set is not independent of the training set.

For image data, the independence of test and training sets requires that images from the same patient (or the same slide, or the same session) are never split across them. When the dataset carries no patient identifier, a common practice is a random split of images, which does not satisfy this requirement. Related to this is shortcut learning (Geirhos et al. 2020): a model may rely on features that correlate with the label in the dataset, such as the source of the images, but that have no causal relation to the disease. DeGrave et al. (2021) showed that classifiers of chest radiographs can learn such shortcuts, and that explanation methods can mislead about it (Adebayo et al. 2018). Blocked cross-validation is a recognised remedy where observations are dependent (Roberts et al. 2017), and estimates of generalisation error from cross-validation are themselves uncertain (Kohavi 1995; Nadeau and Bengio 2003).

Contributions

This paper makes five contributions, all of which are supported by result files released with the code. (1) A documented and tested preprocessing pipeline for smear images with quantitative checks, including a stabilised diffusion denoiser and a procedure that isolates cells from their background. (2) A shortcut audit that measures how much class information is present in the background, the scale bar and the colour of cells. (3) A demonstration, with a purge analysis, that proximity in capture order carries class-relevant information and that random block folds overestimate accuracy. (4) A reproducible baseline with a frozen hematology foundation

model, with intervals computed over groups and calibrated probabilities. (5) A protocol, implemented in code, for fine-tuning and encoder comparison under the same folds.

The remainder of the paper is organised as follows. The Materials and Methods give the data, the preprocessing, the shortcut audit, the evaluation design, the models and the statistical procedures. The Results report the measurements in the order in which they were made. The Discussion interprets the findings, states the limitations and proposes next steps. The appendices give per-fold results, definitions, the configurations and listings of the core algorithms, so that every reported number can be traced and every step repeated.

Objectives

The aim of this work was to establish how accurately the four classes can be separated when the evaluation is designed to prevent leakage between neighbouring images, and how much of the accuracy can be attributed to cell morphology rather than image acquisition. I addressed four questions. (1) Can a reproducible preprocessing pipeline - colour normalisation, denoising, segmentation and per-cell isolation - be built and checked quantitatively? (2) How much of the class information is present in acquisition cues that are unrelated to the cells? (3) How large is the optimism introduced by random block folds, relative to contiguous session-aware folds, and is it caused by proximity in capture order? (4) What accuracy does a frozen foundation-model encoder reach under the strictest protocol that the data permit? I did not attempt to claim clinical performance; the data do not allow it.

MATERIALS AND METHODS

Dataset

I used the 3256 RGB images of the public dataset (Aria et al. 2021; Ghaderzadeh et al. 2022) (Kaggle identifier mehradaria/leukemia). All images are 224 x 224 pixels. The four classes

are Benign (504 images), Early (985), Pre (963) and Pro (804). The segmentation stage found 36167 cells (Table 1). I kept the file names unchanged and recorded a SHA-256 hash for every image. Comparison of SHA-256 hashes found 40 groups of byte-identical images (80 images); duplicates were assigned to the same group so that they could never be separated by a split.

Preprocessing

The preprocessing pipeline (Fig. 1) is a Python package with a command-line interface and a configuration object whose hash is stored with every output. It applies five stages to each image.

Colour normalisation. Stain variation between slides was reduced with the Reinhard method (Reinhard et al. 2001): the mean and standard deviation of each CIE Lab channel are matched to those of a reference computed from a sample of images. Three details differ from a direct implementation. The computation is carried out in floating point to avoid quantisation. Statistics are computed over foreground pixels only, and pixels with L^* below 8 are excluded, because several images contain a black scale bar whose pixels otherwise shift the statistics and change the colour of cells (this was found as a bias in an early version and is covered by regression tests). The target statistics are computed once for the dataset and stored. The influence of colour normalisation on classifiers has been reported by Tellez et al. (2019).

Denoising. Three denoisers are available: bilateral filtering (Tomasi and Manduchi 1998), non-local means, and a diffusion scheme (HDS) that combines Perona-Malik anisotropic diffusion (Perona and Malik 1990) with total-variation regularisation. The original repository implemented the diffusion without a bound on the diffusivity, and the result did not remove noise (Table 2). My version normalises the diffusivity to at most 1, restricts the time step to 0.25 or less, and adds a fidelity term, which makes the iteration bounded; a test with 3000 iterations

confirmed that the output stays finite and inside the input range. The benchmark in Table 2 uses synthetic Gaussian noise added to the dataset images. The full-dataset run reported in the Results used bilateral filtering (diameter 9, colour and space sigma 75); the diffusion denoiser is disabled by default and the benchmark documents the module only.

Cell segmentation. White blood cells stain more strongly in the green-magenta (a) axis than the surrounding red cells and background. I applied Otsu's threshold (Otsu 1979) to the a channel with a floor of 8, removed small objects by morphological opening, filled holes, and separated touching cells with a marker-controlled watershed whose markers are h-maxima of the distance transform (van der Walt et al. 2014). Cells that touch the image border are kept and flagged. In total 1694 images were flagged because no nucleus was resolved and 108 because most of their cells touched the border. An example is shown in Fig. 2. No ground-truth masks are available for this dataset, so the segmentation was checked visually on a sample and with consistency tests, not by overlap with annotations; I make no claim about segmentation accuracy.

Nucleus segmentation. Within each cell, a second threshold was applied to the interior to isolate the nucleus. This step returned a nucleus in only 7.6% of cells overall (Table 1), so nuclear measurements (for example the nucleus-to-cell ratio) are missing for most cells and were not used as primary features. I report this failure rate here instead of imputing the measurements.

Response layers. Three local-structure maps were computed from the normalised image: a Laplacian-of-Gaussian (LoG) response with scale normalisation, a Canny edge map, and an adaptive principal-curvature response (APC), a weighted combination of the absolute Hessian eigenvalues in the manner of the vesselness measure of Frangi et al. (1998). The gains are fixed constants, not data-dependent, so that the values are comparable between images.

Shortcut neutralisation and cell isolation

To remove information that identifies the acquisition session rather than the cell, I produced a "clean" version of each image in which every pixel outside the (dilated) segmented cells is replaced by a neutral grey fill and the cells are white-balanced. Each cell was then cropped and resized for the encoder. A grayscale version of the clean image was also produced. The unprocessed background-visible versions (normalised and raw) were retained for the ablation. The effect of neutralisation was measured by the audit described next, and not assumed.

Shortcut audit

The audit trains a random-forest classifier (Pedregosa et al. 2011) on a single low-dimensional quantity (for example the mean background colour, a patch of the corner containing the scale bar, or the mean cell colour) and reports its balanced accuracy under session-aware folds. If a quantity that contains no information about the cells predicts the class far above chance (0.25), the dataset contains a shortcut. The audit is also applied to the neutralised images, where background and corner probes must return chance, as a check on the neutralisation itself.

Session groups and cross-validation

File names are numbered in the order of capture within each class. If images captured consecutively come from the same slide or the same session, random assignment of images to folds leaks information. I measured this directly: in the Early, Pre and Pro classes, images with adjacent capture numbers had a median feature distance that was 44.1%, 29.2% and 55.9% smaller, respectively, than that of random pairs from the same class (Fig. 3); the Benign class showed no such structure. I therefore assigned images to groups by consecutive blocks of 37 capture numbers (and merged duplicate files). This gives 90 groups. The block length of 37 was fixed before modelling and the effect of changing the embargo is analysed in Results.

Cross-validation used five *contiguous* folds: within each class, the images ordered by capture number are cut into five unbroken stretches, and fold k contains the k -th stretch of every class. A validation stretch is taken from the fold two positions after the test fold, and an *embargo* of 37 capture numbers is removed from the training set on both sides of the test and validation stretches. All preprocessing statistics that depend on the data (the stain reference) are computed independently of labels. Confidence intervals use a cluster bootstrap that resamples whole groups (Efron 1979) (1000 resamples), and paired comparisons between variants use the paired cluster bootstrap with Holm's correction (Holm 1979). The Nadeau-Bengio corrected resampled t statistic (Nadeau and Bengio 2003) is available in the code for comparisons between fold-level results.

Models

The headline model is a frozen DinoBloom-S encoder (a small DINOv2 vision transformer trained on hematology images (Koch et al. 2024; Oquab et al. 2024)), applied to each cell crop at 112 x 112 pixels, followed by gated attention pooling over the cells of an image (Ilse et al. 2018) and a linear classification layer. Only the pooling and classification layers were trained (AdamW (Loshchilov and Hutter 2019), learning rate 0.001, batch size 16, at most 30 epochs, early stopping with patience 8 on validation macro-F1, seed 0). Class imbalance was handled with inverse-square-root class weights in the loss. Probabilities were calibrated by temperature scaling (Guo et al. 2017) fitted on the validation stretch of each fold.

For the ablations and the leakage analysis I used a faster setting in which the frozen embeddings of the cells of an image are averaged and a multinomial logistic regression with balanced class weights is fitted on standardised features. This probe is deterministic given the folds; the three seeds in the tables rotate the boundaries between folds. Alternative encoders

(ResNet (He et al. 2016), EfficientNet (Tan and Le 2019), from the timm library (Wightman 2019); generic DINOv2) and a hybrid model that concatenates hand-made features to the embedding are defined in the code. Low-rank adaptation (Hu et al. 2022) and full fine-tuning are implemented and tested (see Appendix IV) but were not run for this paper.

Statistics and software

All code is in a public repository; the analysis was run on a CPU only, with Python 3.11, PyTorch (Paszke et al. 2019) 2.14, scikit-learn 1.9, OpenCV (Bradski 2000) and scikit-image. The code is covered by 189 automated tests. The pipeline configuration hash of the run reported here is 9136b9beac982264; the model configuration hash is 5e8dd4b86e31ac7c. Every number in this paper is read from a result file in the repository by the script that builds the manuscript.

Formal description of the preprocessing

This section gives the operations of the Materials and Methods in formal terms so that they can be reproduced independently of the code. Images are denoted by I , the CIE Lab representation by (L, a, b) with L in $[0, 100]$ and a and b signed, and the foreground set of an image by F .

Reinhard normalisation. For each image, the set of tissue pixels V contains those with $L \geq 8$ (excluding near-black annotation overlays such as the scale bar). The foreground F is the subset of V whose lightness is below the Otsu threshold of L computed over V . If V covers less than half of the image, the lightness has a standard deviation below 1, or F covers less than the minimum fraction (0.02) of the image, the image is left unchanged and flagged. Otherwise the per-channel mean $m(I)$ and standard deviation $s(I)$ over F are computed. With reference statistics m and s (the mean of the per-image statistics over a sample of 20 images), each channel ch is mapped by

$$ch' = (ch - m_{ch}(I)) * g_{ch} + m_{ch}, \quad g_{ch} = \text{clip}(s_{ch} / \max(s_{ch}(I), 1e-3), 0.25, 4.0)$$

and L' is clipped to $[0, 100]$. The gain g is clipped to the interval $[0.25, 4]$ so that an unusual image cannot be stretched without bound.

Stabilised diffusion. Let f be the observed single-channel image and u the iterate, with $u_0 = f$. For the four neighbours k (north, south, west and east) let d_k be the difference between the neighbour and the central pixel. The diffusivity is a convex combination of a Perona-Malik term and a Charbonnier total-variation term,

$$c(s) = w / (1 + (s/h)^2) + (1 - w) * \beta / \sqrt{s^2 + \beta^2}, \quad 0 < c(s) \leq 1$$

$$u_{t+1} = u_t + dt * (\sum_k c(|d_k|) * d_k - \lambda * (u_t - f))$$

with $w = 0.5$, $h = 0.05$, $\beta = 0.05$, $\lambda = 0.3$, $dt = 0.2$ and at most 30 iterations; iteration stops when the mean absolute update falls below 0.0001. Because c is at most 1 and dt is at most 0.25, each update is a convex combination of the neighbouring values and the observed value, so the iterate cannot leave the range of the input. The unmodified scheme of the original repository has a conduction coefficient with a term of size $1/\epsilon$, which violates the stability bound and explains the loss of signal-to-noise ratio in the benchmark.

Foreground, nucleus and instance segmentation. Let a_s be the a^* channel smoothed with a Gaussian of standard deviation 1 pixel. The cell mask is $a_s > \max(\text{Otsu}(a_s), 8)$. The mask is opened with a disk of radius 1, components smaller than 120 pixels are removed, and holes are filled. Each connected component is split by a marker-controlled watershed on the negative smoothed distance transform, with markers at the h -maxima ($h = 1.5$) of the distance transform; a split is accepted only if every resulting piece has at least 120 pixels. Within each cell, the nucleus is sought only in the interior (the cell eroded by 2 pixels): Otsu's threshold is computed on the smoothed lightness of the interior, and a nucleus is accepted only if the mean

lightness of the two classes differs by at least 8 units, the nucleus has at least 30 pixels and at least 5% of the cell area. When any condition fails, no nucleus is reported for that cell.

Neutralisation. Let K be the cell mask dilated by the configured number of pixels. The background colour b is the median Lab value of the brighter half (by lightness) of the pixels outside K , if at least a minimum number of such pixels exist. The lightness of every pixel is multiplied by a gain, clipped to a fixed interval, that brings the background lightness to the neutral fill lightness, and b^* values of a and b are subtracted from the corresponding channels. Pixels outside K are then set to the neutral colour (fill lightness, $a = b = 0$). The grayscale variant keeps only $L/100$.

Cell crops. For every cell, a square window is cut around the bounding box of the cell, enlarged by a margin of 25% on each side. Where the window leaves the image, it is padded with the neutral fill colour. When the isolation option is on, all pixels except those of this cell, dilated by 2 pixels, are replaced by the fill colour, so that neighbouring cells do not appear in the crop. The window is resized to 128 x 128 pixels for storage and to 112 x 112 pixels for the encoder (a multiple of the 14-pixel patch size of the vision transformer).

Bag construction and the attention head

Each image is a *bag* of cells. During training, at most 24 cells per image (16 in the GPU configurations) are drawn at random; at evaluation, up to 64 cells are taken in order of their label number. For the cells $h_1, \dots, h_K$ of a bag, with embeddings of dimension D (384 for DinoBloom-S), the head computes $z_k = \text{GELU}(W h_k + b)$ with 256 hidden units and dropout 0.2, and the attention weights

$$a_k = \frac{\exp(w^T [\tanh(V z_k) * \text{sigmoid}(U z_k)])}{\sum_j \exp(w^T [\tanh(V z_j) * \text{sigmoid}(U z_j)])}$$

$$\text{bag} = \sum_k a_k z_k ; \quad \text{logits} = C \text{ bag}$$

where the softmax runs over the valid cells of the bag only and the attention dimension is 128 (Ilse et al. 2018). Because the weights sum to 1, the bag representation does not grow with the number of cells, which matters because the number of cells per image differs strongly between classes (Table 1). The head has a small number of trainable parameters compared with the encoder, which is frozen.

Training objective and optimisation

The loss is the cross-entropy with label smoothing of 0.1 and with class weights proportional to the inverse square root of the class frequency of the training images. Optimisation uses AdamW (Loshchilov and Hutter 2019) (an improved variant of Adam (Kingma and Ba 2015)) with weight decay 0.05, gradient clipping at a norm of 1, a linear warm-up over the first 10% of the steps and a cosine decay of the learning rate to zero (Fig. 4c). For the frozen encoder, embeddings of all cells are computed once and cached, so no augmentation is applied; the augmentation modules (flips, 90-degree rotations, scale jitter, stain-colour jitter restricted to non-fill pixels, random grayscale and blur) are used only when the encoder is adapted. Training stops when the validation macro-F1 has not improved for 8 epochs, and the weights of the best epoch are used. The temperature of the final softmax is fitted by minimising the validation negative log-likelihood (Methods, Evaluation measures).

Fold construction in detail

Let each image i have a class y_i and a capture number n_i , the number in its file name. For each class, the images are sorted by n and cut into five consecutive segments of nearly equal size. Fold k uses segment k of every class as the test set and the segment two positions later (modulo five) as the validation set. The training set contains the remaining images, from which every image of the same class whose capture number is within 37 of any test or validation image

of that class is removed (the embargo). Consecutive segments therefore share at most two boundaries per class and fold, and images in a boundary region are excluded. The random-block scheme used for comparison draws 37-image blocks (merging duplicates) and assigns whole blocks to folds with class stratification.

For the purge analysis, the training images that lie within a distance e of any test image of the same class are removed from a random-block training set, for e in $\{0, 37, 75, 150\}$. The control removes, for each class, the same number of training images chosen uniformly at random. For the gap analysis, the embargo of the contiguous scheme is increased to e in $\{0, 37, 75, 150, 225, 300\}$, and the control again removes the same number of images per class at random.

Evaluation measures

Balanced accuracy is the mean of the per-class recalls and is the primary measure because the classes are unequal in size (Sokolova and Lapalme 2009). Macro-F1 is the unweighted mean over classes of the harmonic mean of precision and recall. The AUROC is the macro average of one-vs-rest areas under the receiver operating characteristic computed from the rank statistic. The expected calibration error (ECE) is the weighted mean absolute difference between accuracy and mean confidence over 15 equal-width confidence bins. Sensitivity and specificity for the benign-versus-malignant decision treat all predictions of Early, Pre or Pro as positive. All measures are computed from the out-of-fold predictions of the five folds pooled, and also within each fold.

Intervals use the cluster bootstrap over the 90 groups with 1000 resamples and the 2.5 and 97.5 percentiles; a paired comparison resamples the same groups for both variants. The p value of a paired comparison is twice the smaller tail proportion of the bootstrap difference and

cannot be smaller than the reciprocal of the number of resamples; Holm's correction is applied over the family of comparisons within each table.

Hyperparameters of the planned fine-tuning experiments

Eight fine-tuning configurations were prepared. They share the data, fold and optimisation settings and differ in the encoder, the adaptation method or the input, as listed in Table A13 (Appendix VIII). The adaptation uses low-rank updates of rank 8 and scaling 16 on the query-key-value and output projections of every attention block of the vision transformers (Hu et al. 2022); the full fine-tuning configurations update all weights of the convolutional network with a smaller learning rate than the head.

Computational environment

All reported analyses were run on a CPU. Preprocessing of the 3256 images took 10.5 minutes. Cross-validation of the headline model took 33 minutes for the five folds (the encoder embeddings are computed once and cached). The analyses were run in an isolated container without a graphics processor; the corresponding code paths for graphics processors were tested only for correctness of the logic on a CPU, and the associated risks are stated in the Discussion.

RESULTS

Dataset and preprocessing

The pipeline processed all 3256 images without failure in 10.5 min on a CPU. It produced 36167 cells, a mean of 11.1 cells per image; the Pre class contained many more cells per image (20.5) than the other classes (7.9, 7.5 and 6.2 for Benign, Early and Pro; Table 1). Between 22.6% and 30.0% of the cells in a class touched the image border. Example images of the four classes are shown in Fig. 5.

For synthetic noise with a standard deviation of 0.05-0.12, the stabilised diffusion improved the peak signal-to-noise ratio by 12.3 dB on average, comparable with bilateral filtering (11.2 dB) and above Gaussian smoothing (4.1 dB), whereas the original scheme made the image worse (-17.2 dB; Table 2, Fig. 6). Stabilised diffusion removed more texture than bilateral filtering did (correlation with the noisy image's texture 0.37 versus 0.29, from 0.50 for the noisy input).

The class can be predicted from the background

The audit showed strong acquisition cues in the raw images (Table 3). The mean background colour alone, three numbers, gave a balanced accuracy of 0.823 (chance 0.25), and the scale-bar corner patch gave 0.716. After neutralisation, the background probe fell to chance (0.250), confirming that the neutralised images no longer carry the background. The corner patch of the neutralised image still gave 0.435, above chance, which I attribute to the small residual pattern left by the dilated mask at the image edge.

The colour of the cells themselves is not free of session information: the mean colour of the white-balanced cells gave 0.798. A random forest on all aggregated hand-made features reached 0.909, whereas shape and texture features without colour or density reached 0.776 (Table 3). Cell colour is therefore the most informative single group, and it cannot be separated from stain and session effects in this dataset.

Headline result with a frozen foundation model

With the headline model (Table 4, Fig. 7), the pooled out-of-fold balanced accuracy was 0.968 (95% CI 0.955-0.978) and the macro-F1 was 0.967 (95% CI 0.950-0.978). The AUROC (one-vs-rest) was 0.995. The recall of the four classes was Benign 0.960, Early 0.973, Pre 0.957, Pro 0.980. Treating the three malignant classes as one, sensitivity was 0.990 and specificity

0.960. The per-fold balanced accuracy ranged from 0.943 to 0.989. The most frequent confusions were Pre predicted as Early (16 images), Pre predicted as Pro (15) and Early predicted as Benign (14). Training stopped after 12 to 23 epochs per fold and took 33 min for all folds on a CPU.

Calibration was poor before correction and good after: the uncalibrated expected calibration error was 0.121 on average over folds, and temperature scaling reduced the pooled value to 0.010. The fitted temperatures ranged from 0.24 to 0.55; a temperature below 1 means the raw probabilities were under-confident.

What the model uses: representation ablation

Table 5 and Fig. 8 compare five inputs under the same protocol. Isolated cells with a neutral background (A) gave a balanced accuracy of 0.959 (95% CI 0.944-0.973). Removing stain colour (B, grayscale) lowered it by 0.018 (95% CI 0.010-0.027, Holm $p = 0.004$), so colour contributes a measurable but small part of the accuracy. Giving the model the background as well, after stain normalisation (C) or raw (D), raised the accuracy to 0.975 and 0.978; both differences from A were significant (Holm $p = 0.004$ for each). The hand-made shape, texture and curvature features alone (E, no pixels) gave 0.825, well below the encoder.

The same ablation was first computed with random-block folds and was superseded by the contiguous result: the accuracy of variants A-E was 0.969, 0.959, 0.988, 0.992, 0.854, respectively, and the random-block results are kept in the repository for transparency.

Random folds overestimate accuracy, and proximity is the cause

With DinoBloom-S features, random 37-image blocks gave a balanced accuracy of 0.969 and contiguous folds gave 0.963 with no embargo and 0.959 with an embargo of 37 (Table 6). For hand-made features the gap was larger: 0.854 for random blocks versus 0.825 for contiguous folds with the embargo. The difference between the schemes is therefore about 1.0 percentage

points for the foundation-model features and 3.0 for the hand-made features. A larger embargo reduced accuracy slightly further (0.953 at 75).

If proximity in capture order carries information, removing near neighbours of the test images from the training set should hurt more than removing the same number of images at random. This was the case (Table 7, Fig. 9a). With random-block folds, purging training images within 75 capture numbers of the test images (leaving 954 training images) gave 0.930, whereas removing 954-sized random subsets gave 0.965. At a purge distance of 150 the purged model fell to 0.744 against 0.954 for random removal. The same pattern appeared with contiguous folds (Table 8, Fig. 9b): at a gap of 225 the accuracy was 0.932 (random removal 0.960) for DinoBloom-S and 0.761 (random 0.829) for the hand-made features, which depend on proximity more strongly. At a gap of 300 the training sets were small and class coverage was reduced, and accuracy fell also with random removal (0.901), so this setting does not isolate the effect of proximity.

These purge and gap analyses are single runs per setting, without confidence intervals, and I interpret them as a consistent pattern rather than as precise estimates.

Cell features by class

Table 9 summarises the per-cell measurements for the 26202 cells that do not touch the border. Cell size differed between classes: the median area was 360 px for Benign, 486 for Early, 447 for Pre and 401 for Pro, with the Benign cells the smallest. Measures of the outline - aspect ratio, solidity and circularity - were close to identical in the four classes (for example, the median solidity was 0.95 to 0.96 in all of them), so that the shape of the outline of a segmented cell carries little class information in this dataset. Texture measures differed modestly.

The colour of the cells differed much more. The median a value was 22.2 in *Pre cells* and 35.8 in *Pro cells*, with 28.2 in *Benign* and 34.1 in *Early cells*, and the median b value ranged from -37.2 (*Pre*) to -50.7 (*Early*). The *Pre* class stands out from the other three in lightness and in both chromatic channels. Because the classes are not expected to differ so strongly in the colour of their cytoplasm and chromatin, and because the colour statistics were computed after stain normalisation, this pattern is consistent with the class-specific acquisition conditions that the shortcut audit detected. It cannot be separated from a true biological difference with these data.

Training dynamics of the headline model

Training the attention head converged quickly (Fig. 4). The mean training loss over the five folds fell from 0.99 in the first epoch to 0.38 in the last, and most of the decrease occurred within the first three epochs. The validation balanced accuracy of the best epoch was between 0.960 and 0.993 across folds, and the best epoch was reached after 4 to 15 epochs. The runs stopped after 12 to 23 epochs by the early-stopping rule, much earlier than the maximum of 30. The validation score of the folds differed by several points at the same epoch (Fig. 4b), which reflects that validation stretches contain different sessions and is the same variation that the per-fold results show.

Discrimination and precision

The one-vs-rest area under the ROC curve of the pooled out-of-fold probabilities was 0.996 for *Benign*, 0.995 for *Early*, 0.988 for *Pre* and 0.999 for *Pro* (Fig. 10a). The area under the precision-recall curve was 0.986, 0.984, 0.986 and 0.993 (Fig. 10b). The *Pre* class had the lowest area under the ROC curve although its precision-recall area was similar to that of the other classes, which indicates that it is confused with its neighbours in the ordering of the disease

stages without being poorly ranked overall. These values are computed from probabilities pooled over the folds, so they combine models that were fitted on different training sets.

Calibration

Before temperature scaling the probabilities of the model were under-confident: the expected calibration error averaged 0.121 over folds, and the fitted temperature was below 1 in every fold (range 0.24-0.55). After scaling, the pooled error was 0.010 (Fig. 11a). The median confidence was 0.998 for correct and 0.811 for incorrect predictions (Fig. 11b), so the confidence can be used to flag uncertain images for review. A temperature fitted on one stretch of one dataset is not guaranteed to transfer, and the calibration should be re-checked on any new data.

Where the model fails

Accuracy depended on the number of cells in the image (Fig. 12a). Images in which the pipeline found one to three cells were classified correctly in 87.4% of cases ($n = 294$), against 98.4% for six to eight cells ($n = 929$) and 98.6% for nine to twelve ($n = 565$). Images with few cells provide the model with little evidence, and some of them may contain cells that the segmentation missed; I did not check this image by image. The largest attention weight of an image had a median of 0.24, which is about 1.9 times the weight that uniform attention would give (Fig. 12b), so the head weights cells unequally but does not rely on a single cell. In total 104 of 3256 images were misclassified; Appendix VI lists them with their true class, predicted class and confidence.

The errors were concentrated between neighbouring stages. Of the 27 Early images that were misclassified, 14 were called Benign, 7 Pre and 6 Pro. Of the 41 misclassified Pre images, 10 were called Benign, 16 Early and 15 Pro. Benign images were called Early (9), Pre (10) or Pro (1) in 20 cases, and 16 of 804 Pro images were misclassified. Because the dataset labels the

stages of a continuous maturation process, confusion between neighbouring stages is expected even for expert observers; I did not have access to a second reading of the images and cannot quantify human agreement.

Fine-tuning and encoder comparison

The preceding results use a frozen encoder. Adapting the encoder to the task can improve the fit to the cell appearance, but it also gives the model more capacity to learn the acquisition cues that the audit identified. Eight configurations were therefore defined (Methods, Hyperparameters of the planned fine-tuning experiments): low-rank adaptation of DinoBloom-S with colour (g01), in grayscale (g02) and with a feature branch (g03); full fine-tuning of two convolutional networks (g04, g05); low-rank adaptation of a generic DINOv2 model (g06) and of the larger DinoBloom-B (g07); and adaptation with the background left in the image (g08), which is a deliberate shortcut control. Table 10 lists the results of the runs that have been completed.

[[GPU: 8 of 8 configurations have no result file yet. Write here: (1) the balanced accuracy of g01 compared with the frozen reference, with the paired-bootstrap difference and Holm-adjusted p value; (2) the effect of colour (g01 vs g02); (3) the effect of the feature branch (g03 vs g01); (4) the ordering of the encoders (g01, g04, g05, g06, g07); (5) the shortcut control (g08 vs g01). Delete this paragraph if the runs are not performed.]]

Figure 13 shows the training curves of the adapted model and Fig. 14 compares the configurations with their intervals. The confusion matrix of the best configuration is shown in Fig. 15.

Attention, saliency and embedding structure

Two further analyses address what the adapted models use. Attention weights over the cells of an image (Methods) indicate which cells drive the bag representation, and gradient-weighted class activation maps (Selvaraju et al. 2017) indicate which regions of a cell drive the score. Because saliency maps can look plausible for untrained networks, a model-randomisation check (Adebayo et al. 2018) is part of the protocol. Fig. 16 is reserved for the examples. A two-dimensional projection of the embeddings coloured by class and by capture segment (Fig. 17) addresses the question of whether the embedding encodes the session as strongly as the class.

[[GPU: after running scripts/analysis_xai.py on a checkpoint, describe in two to three paragraphs what the attention and saliency maps show, including at least one failure case; report the model-randomisation result. Delete this paragraph and the two figures above if the analysis is not performed.]]

Table 1. Composition of the dataset and output of the segmentation stage, by class. Cells are connected components of the foreground mask after watershed splitting (Methods, Cell segmentation); "Nucleus found" is the fraction of cells in which the nucleus step returned a mask.

Class	Images	Cells	Cells per image (mean +/- SD)	Median cell area (px)	Border-touching cells (%)	Nucleus found (%)
Benign	504	3972	7.9 +/- 3.5	357	25.4	16.2
Early	985	7404	7.5 +/- 3.0	472	25.7	7.3
Pre	963	19775	20.5 +/- 8.8	435	30.0	5.6
Pro	804	5016	6.2 +/- 2.6	392	22.6	9.1
All	3256	36167	-	432	-	7.6

Table 2. Denoising benchmark on synthetic Gaussian noise (peak signal-to-noise ratio in dB, higher is better). "Legacy HDS" is the diffusion scheme of the original repository; "stabilised HDS" is the bounded re-implementation used here.

Noise SD	Noisy input	Gaussian	Bilateral	Legacy HDS	Stabilised HDS	Stabilised HDS iterations
0.05	26.1	26.4	38.2	5.0	37.5	23
0.08	21.9	26.3	34.0	5.0	34.6	27
0.12	18.4	26.0	27.7	5.0	31.4	30

Mean gain over the noisy input: legacy HDS -17.2 dB, stabilised HDS +12.3 dB, bilateral +11.2 dB, Gaussian +4.1 dB.

Table 3. Shortcut audit: balanced accuracy of a random-forest classifier that sees only the named quantity, under 5-fold session-aware cross-validation (three seeds; chance = 0.25). Verdicts are the audit tool's own thresholds.

Group	Information given to the classifier	Balanced accuracy	SD over seeds	Verdict
acquisition	background colour, raw image (3 numbers)	0.823	0.000	VERY STRONG
acquisition	background colour, after stain normalisation (3 numbers)	0.784	0.001	VERY STRONG
acquisition	scale-bar corner patch, raw image	0.716	0.002	VERY STRONG
after shortcut removal	background colour of rgb_clean (sanity: constant)	0.250	0.000	none (near chance)
after shortcut removal	corner patch of rgb_clean (sanity: constant)	0.435	0.002	weak
residual	mean cell colour, stain-normalised (3 numbers)	0.766	0.002	VERY STRONG
residual	mean cell colour, white-balanced rgb_clean (3 numbers)	0.798	0.002	VERY STRONG
tabular baseline	density (n_cells, wbc_area_frac)	0.588	0.001	-
tabular baseline	cell shape	0.685	0.001	-
tabular baseline	cell texture (GLCM, entropy)	0.647	0.003	-
tabular baseline	cell colour	0.835	0.002	-
tabular baseline	shape + texture (no colour, no density)	0.776	0.001	-
tabular baseline	ALL aggregated features	0.909	0.001	-

Table 4. Headline result: frozen DinoBloom-S with an attention-pooling head on isolated, shortcut-neutralised cell crops. Out-of-fold predictions of five contiguous session-aware folds (embargo 37) are pooled; the interval is a 95% cluster-bootstrap interval over groups.

Metric	Pooled	95% CI	Per-fold mean +/- SD
Balanced accuracy	0.968	0.955-0.978	0.968 +/- 0.019
Macro-F1	0.967	0.950-0.978	0.967 +/- 0.024
AUROC (one-vs-rest)	0.995	-	0.996 +/- 0.004
Sensitivity, malignant vs benign	0.990	-	-
Specificity, malignant vs benign	0.960	-	-
ECE, uncalibrated (per-fold mean)	-	-	0.121 +/- 0.017
ECE, temperature-scaled (pooled)	0.010	-	-

Sensitivity and specificity treat the three malignant classes (Early, Pre, Pro) as positive and Benign as negative.
ECE: expected calibration error.

Table 5. Effect of the image representation on four-class balanced accuracy with a frozen DinoBloom-S encoder and a logistic head (five contiguous folds per class, embargo of 37 capture numbers, three seeds). Intervals are 95% cluster-bootstrap intervals over the 90 session groups; differences are paired against variant A and the p value is Holm-adjusted over the four comparisons.

Variant	Balanced accuracy	95% CI	Macro- F1	Difference vs A (95% CI)	p (Holm)
A. clean RGB, isolated cell (reference)	0.959	0.944-0.973	0.958	-	-
B. clean grayscale, isolated cell (no stain colour)	0.940	0.923-0.958	0.939	-0.018 (-0.027, -0.010)	0.004
C. normalised RGB, background visible	0.975	0.964-0.987	0.973	+0.016 (+0.007, +0.028)	0.004
D. raw RGB, background visible	0.978	0.967-0.988	0.977	+0.019 (+0.010, +0.029)	0.004
E. tabular shape+texture+curvature (no pixels)	0.825	0.797-0.852	0.818	-0.135 (-0.158, -0.114)	0.004

Table 6. The same classifier evaluated under different fold constructions (three seeds; logistic head; mean over folds). "Random blocks" assigns consecutive runs of 37 capture numbers to folds at random; "contiguous" gives each fold one unbroken stretch of capture numbers per class, with the stated embargo removed from the training set on both sides.

Fold construction	Features	Training images	Balanced accuracy	SD over seeds
random 37-image blocks	DinoBloom-S	2605	0.969	0.004
random 37-image blocks	hand-made features	2605	0.854	0.003
contiguous folds, embargo 0	DinoBloom-S	2605	0.963	0.001
contiguous folds, embargo 0	hand-made features	2605	0.837	0.004
contiguous folds, embargo 37	DinoBloom-S	2329	0.959	0.000
contiguous folds, embargo 37	hand-made features	2329	0.825	0.002
contiguous folds, embargo 75	DinoBloom-S	2057	0.953	0.000
contiguous folds, embargo 75	hand-made features	2057	0.812	0.001

Table 7. Training images close in capture number to the test images are removed from random-block folds (purged), compared with removing the same number of training images of each class at random. Mean over folds and two seeds; single run per setting.

Purge distance	Training images	Purged: balanced acc.	Random removal: balanced acc.
0	2605	0.973	0.972
37	1620	0.960	0.969
75	954	0.930	0.965
150	361	0.744	0.954

Table 8. Contiguous folds with an increasing minimum gap between the training images and the test segment ("gap"), compared with removing the same number of training images at random ("random"). Balanced accuracy, mean over five folds, single run per setting.

Gap	Training images	DinoBloom-S, gap	DinoBloom-S, random	Hand-made, gap	Hand-made, random
0	2605	0.964	0.964	0.833	0.831
37	2368	0.959	0.964	0.824	0.832
75	2125	0.953	0.960	0.813	0.829
150	1665	0.944	0.961	0.786	0.830
225	1274	0.932	0.960	0.761	0.829
300	944	0.843	0.901	0.699	0.765

Table 9. Per-cell features by class: median and interquartile range (in parentheses) over cells that do not touch the image border. The statistics are descriptive; cells of the same image are not independent, so no significance tests are given.

Feature	Benign (n = 2964)	Early (n = 5502)	Pre (n = 13852)	Pro (n = 3884)
Cell area (px)	360 (205-552)	486 (426-550)	447 (352-565)	401 (346-459)
Equivalent diameter (px)	21.4 (16.2-26.5)	24.9 (23.3-26.5)	23.9 (21.2-26.8)	22.6 (21.0-24.2)
Aspect ratio	1.38 (1.26-1.57)	1.36 (1.25-1.49)	1.36 (1.28-1.48)	1.38 (1.27-1.52)
Solidity	0.95 (0.93-0.96)	0.96 (0.95-0.97)	0.96 (0.95-0.97)	0.96 (0.96-0.97)
Circularity	0.92 (0.82-0.96)	0.93 (0.89-0.96)	0.94 (0.89-0.96)	0.94 (0.91-0.97)
Mean L*	66.1 (61.1-70.1)	62.7 (59.9-65.1)	68.5 (65.7-71.3)	68.4 (65.1-71.4)
Mean a*	28.2 (21.6-36.4)	34.1 (30.3-38.2)	22.2 (19.1-25.2)	35.8 (32.4-39.7)
Mean b*	-43.7 (-53.3--36.7)	-50.7 (-55.5--46.5)	-37.2 (-41.4--32.9)	-46.5 (-51.4--42.3)
GLCM contrast	3.07 (1.37-5.54)	2.85 (2.20-3.66)	2.27 (1.73-3.08)	2.60 (1.91-3.64)
GLCM homogeneity	0.53 (0.45-0.65)	0.55 (0.51-0.59)	0.57 (0.53-0.61)	0.57 (0.54-0.61)
Entropy	2.78 (1.98-3.19)	2.76 (2.55-2.97)	2.59 (2.41-2.79)	2.79 (2.59-3.02)
Mean curvature response	0.23 (0.17-0.34)	0.24 (0.20-0.28)	0.22 (0.17-0.28)	0.21 (0.18-0.26)
Mean LoG response	0.30 (0.21-0.46)	0.32 (0.26-0.38)	0.27 (0.19-0.35)	0.28 (0.23-0.35)

Table 10. Fine-tuning and encoder comparison under the same five contiguous session-aware folds (embargo 37). Entries marked [[GPU]] are produced by the graphics-processor experiments and are inserted from the result files by the build script; the headline model is shown for reference.

Run	Configuration	Balanced accuracy	95% CI	Macro-F1	AUROC
ref.	DinoBloom-S, frozen encoder, attention head (headline model)	0.968	0.955-0.978	0.967	0.995
g01	DinoBloom-S, low-rank adaptation (RGB, isolated cells)	[[GPU]]	[[GPU]]	[[GPU]]	[[GPU]]
g02	DinoBloom-S, low-rank adaptation (grayscale)	[[GPU]]	[[GPU]]	[[GPU]]	[[GPU]]
g03	DinoBloom-S, low-rank adaptation + hand-made feature branch	[[GPU]]	[[GPU]]	[[GPU]]	[[GPU]]
g04	ResNet-50, full fine-tuning	[[GPU]]	[[GPU]]	[[GPU]]	[[GPU]]
g05	EfficientNet-B0, full fine-tuning	[[GPU]]	[[GPU]]	[[GPU]]	[[GPU]]
g06	DINOv2-S (generic), low-rank adaptation	[[GPU]]	[[GPU]]	[[GPU]]	[[GPU]]
g07	DinoBloom-B, low-rank adaptation	[[GPU]]	[[GPU]]	[[GPU]]	[[GPU]]
g08	DinoBloom-S, low-rank adaptation, raw image with background	[[GPU]]	[[GPU]]	[[GPU]]	[[GPU]]

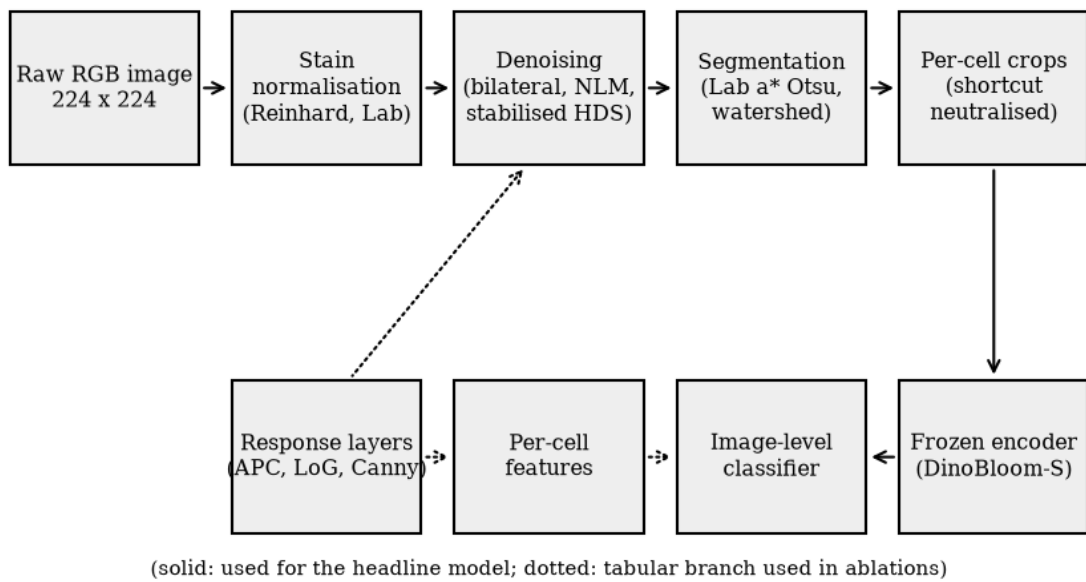


Figure 1. Processing pipeline. Solid arrows show the path used for the headline model; the dotted branch produces the hand-made (tabular) features used in the ablations and the hybrid variant.

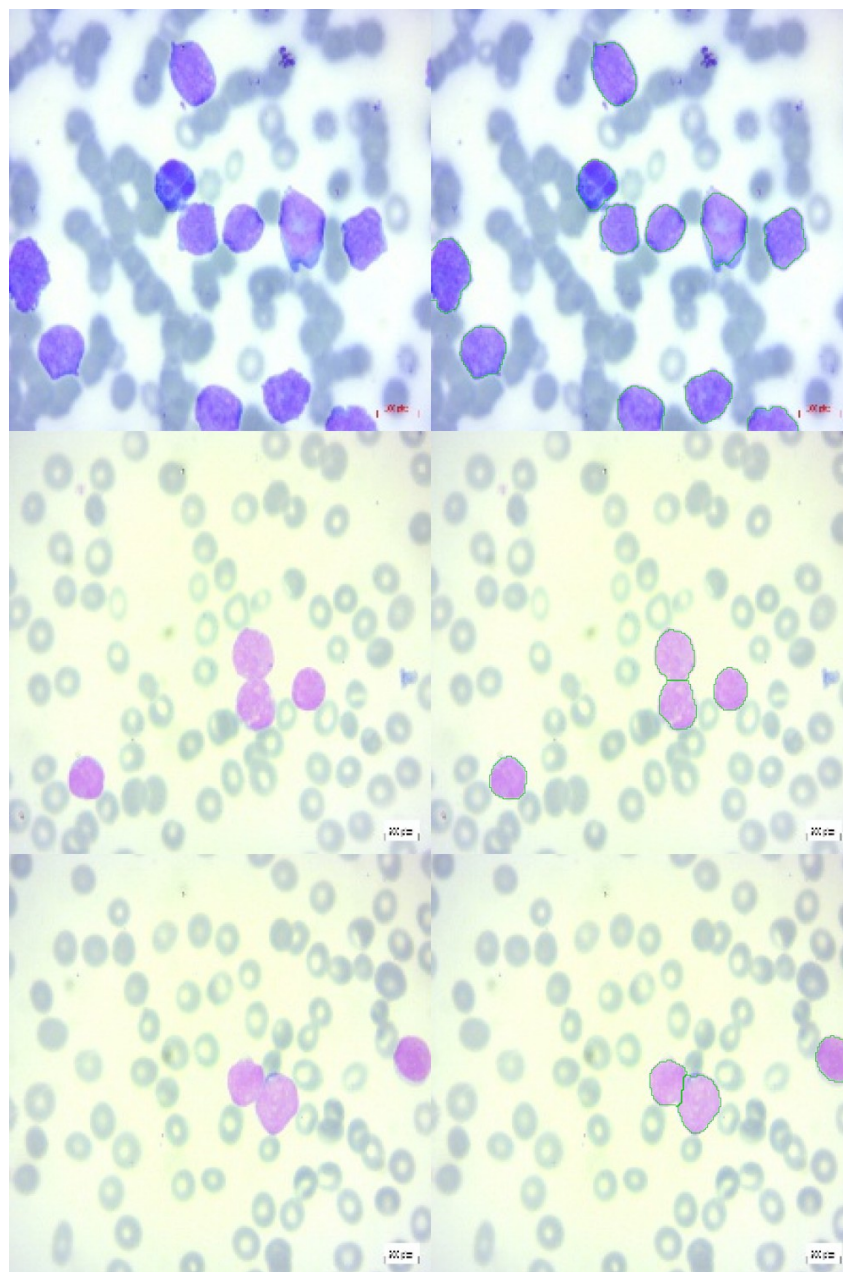


Figure 2. Cell segmentation. Left column: stain-normalised input; right column: detected cell boundaries (green). Touching cells are split by the watershed step (middle row). Border-touching cells are retained but flagged.

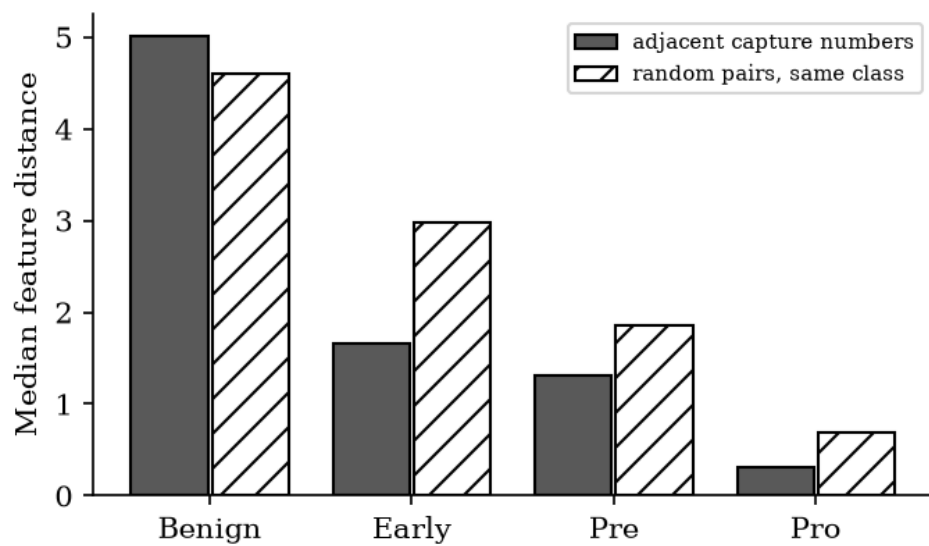


Figure 3. Capture-order structure of the dataset. Median distance between the feature vectors of images with adjacent capture numbers (dark bars) and between random pairs from the same class (hatched bars). Adjacent images are more alike than random pairs in the Early, Pre and Pro classes.

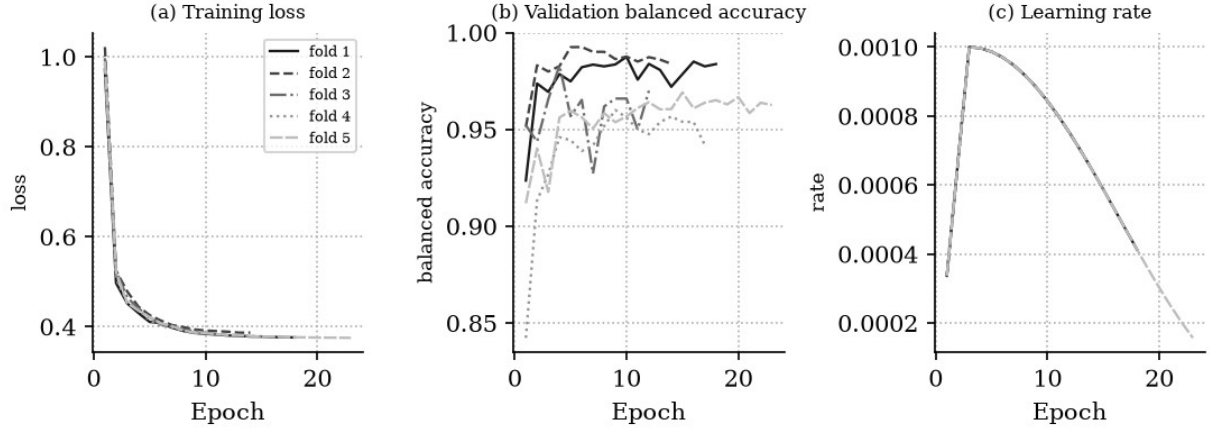


Figure 4. Training of the attention head (frozen DinoBloom-S) in each of the five folds. (a) Training loss (label-smoothed, class-weighted cross-entropy), (b) validation balanced accuracy and (c) learning rate (linear warm-up followed by cosine decay) by epoch. Training of each fold stopped early when the validation macro-F1 had not improved for eight epochs.

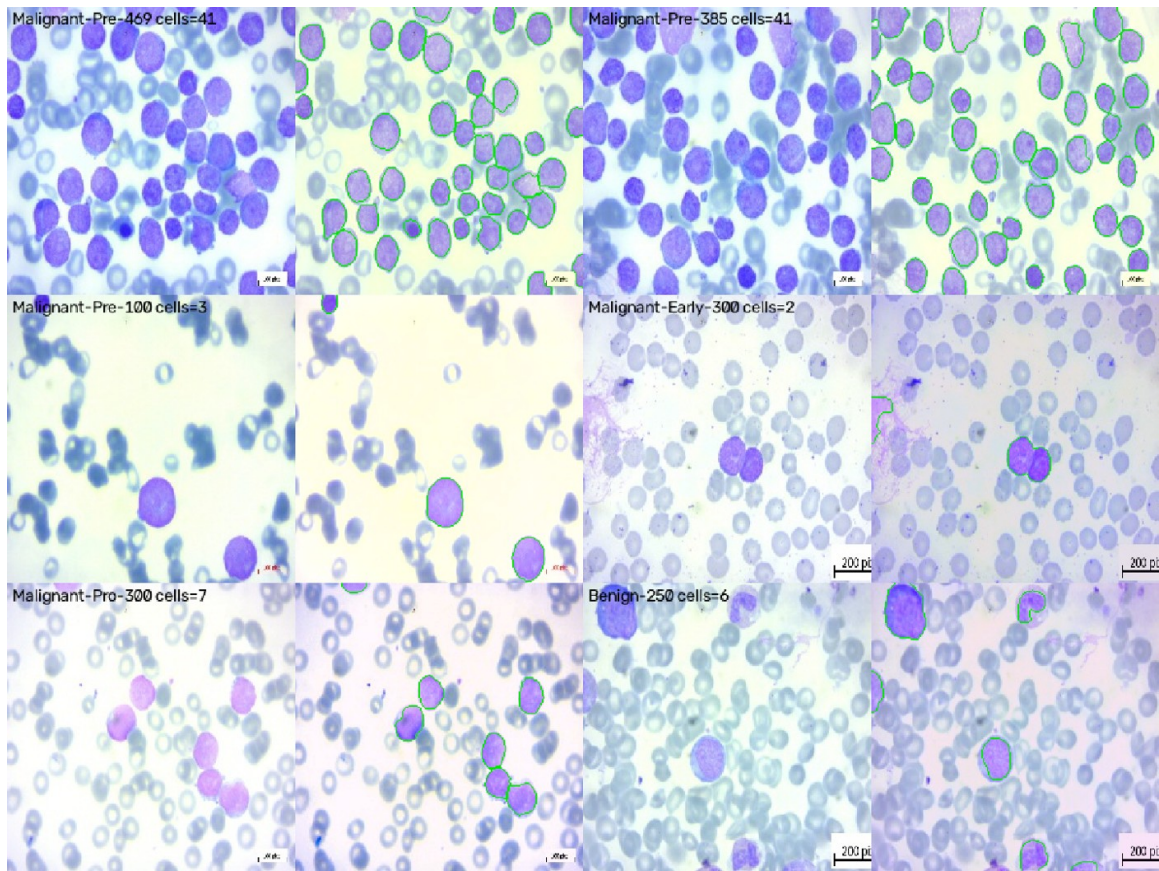


Figure 5. Example images of the four classes (Benign, Early, Pre, Pro) of the dataset, 224 x 224 pixels as supplied. Differences in background colour and illumination between classes are visible.

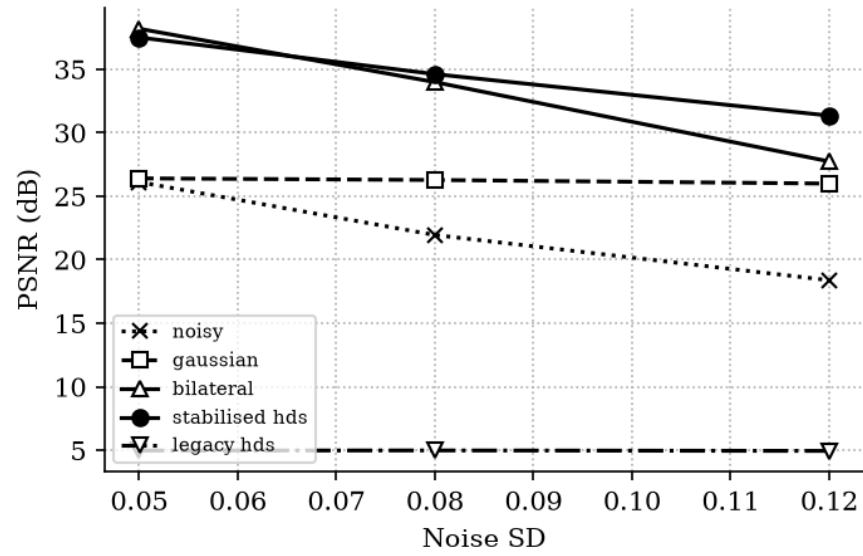


Figure 6. Denoising benchmark (peak signal-to-noise ratio, dB) on synthetic Gaussian noise; stabilised diffusion (solid circles) compared with the unmodified diffusion scheme (inverted triangles), bilateral and Gaussian filtering.

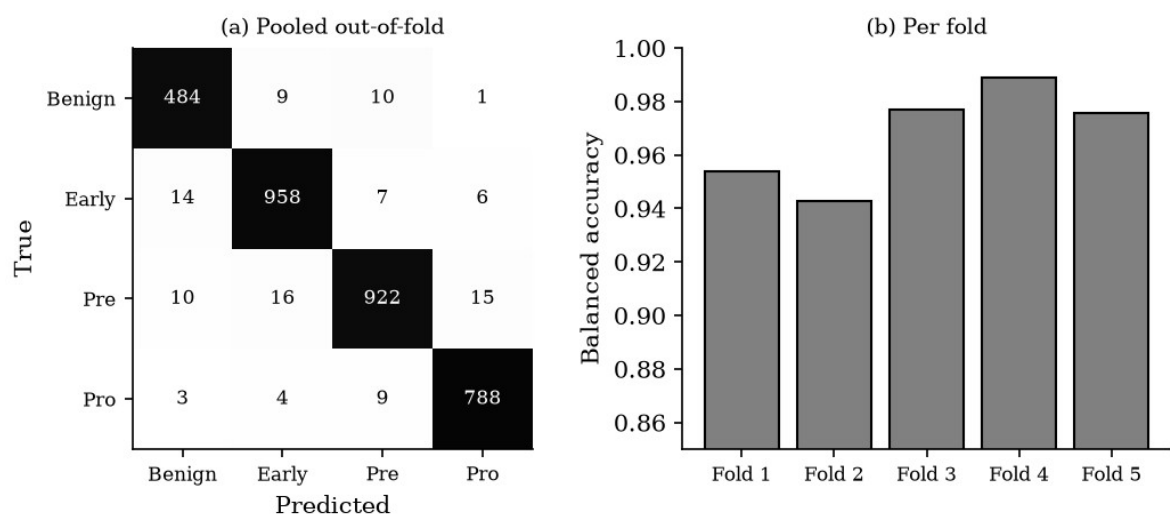


Figure 7. (a) Pooled out-of-fold confusion matrix (counts; shading shows row-normalised proportion) and (b) balanced accuracy of each of the five folds for the headline model.

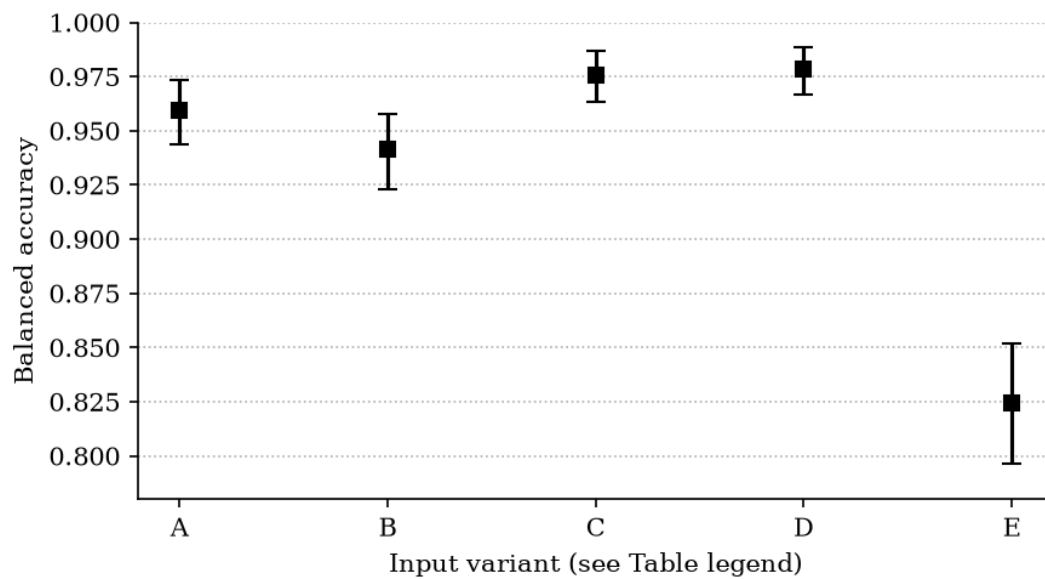


Figure 8. Balanced accuracy of the five input representations (A-E, defined in Table 5), with 95% cluster-bootstrap intervals over session groups. Contiguous folds, embargo 37.

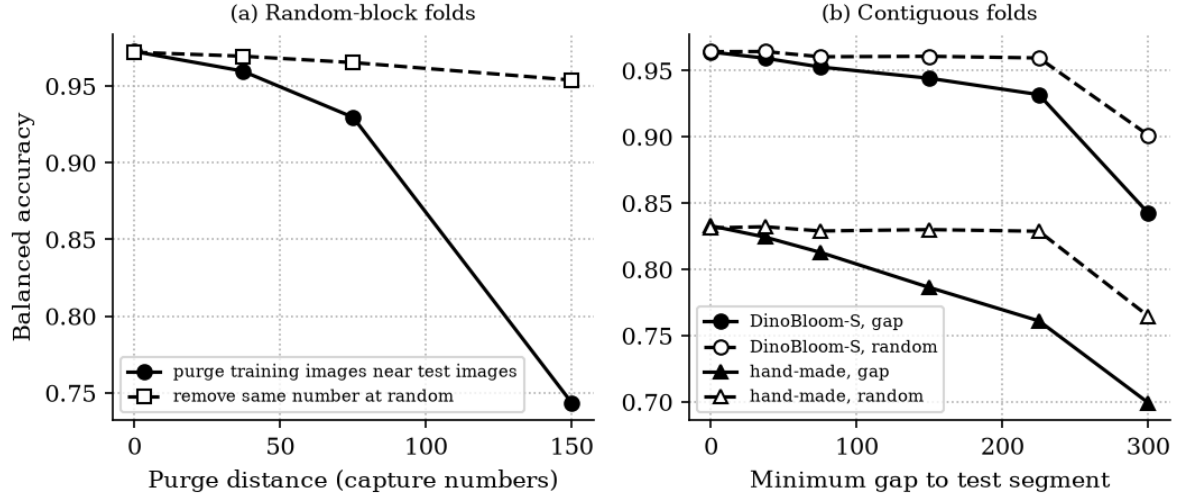


Figure 9. Dependence of accuracy on proximity in capture number. (a) Random-block folds: purging training images near the test images (solid) versus removing the same number of training images at random (dashed). (b) Contiguous folds: increasing the gap to the test segment (solid) versus random removal (dashed), for DinoBloom-S features and for hand-made features.

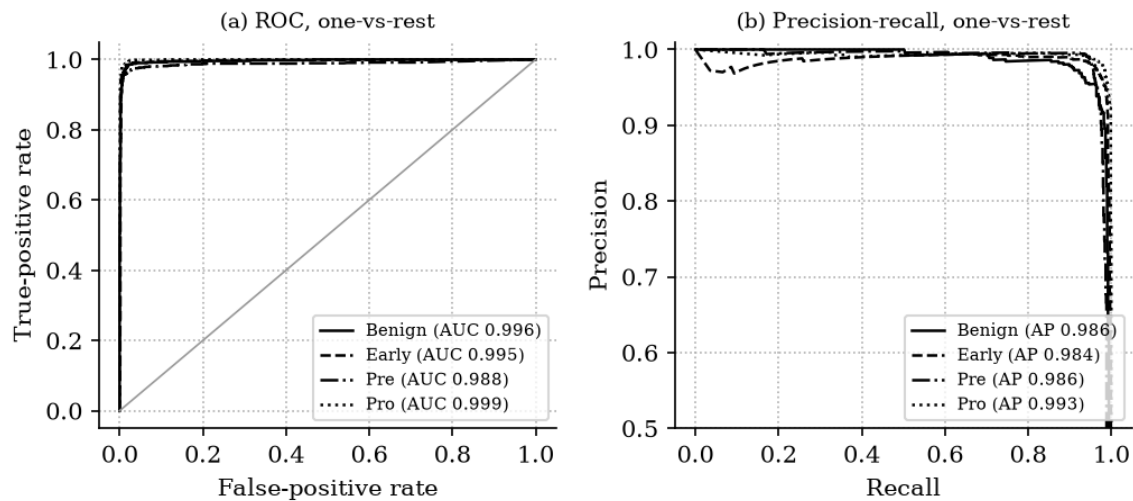


Figure 10. Discrimination of each class from the other three on the pooled out-of-fold predictions. (a) Receiver operating characteristic curves and (b) precision-recall curves of the headline model, with the area under each curve in the legend.

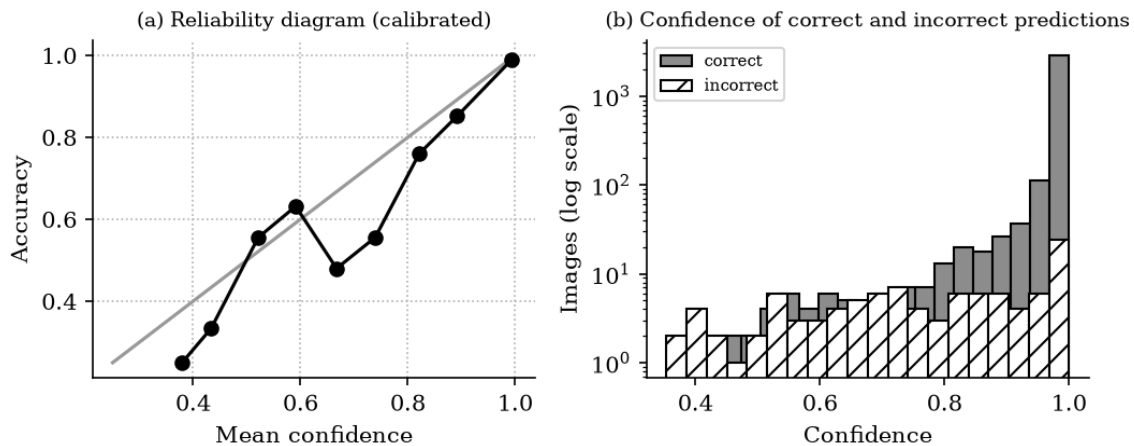


Figure 11. Calibration of the temperature-scaled probabilities on the pooled out-of-fold predictions. (a) Reliability diagram: observed accuracy against mean confidence in ten confidence bins; the diagonal is perfect calibration. (b) Histogram of the confidence of correct and incorrect predictions (log scale).

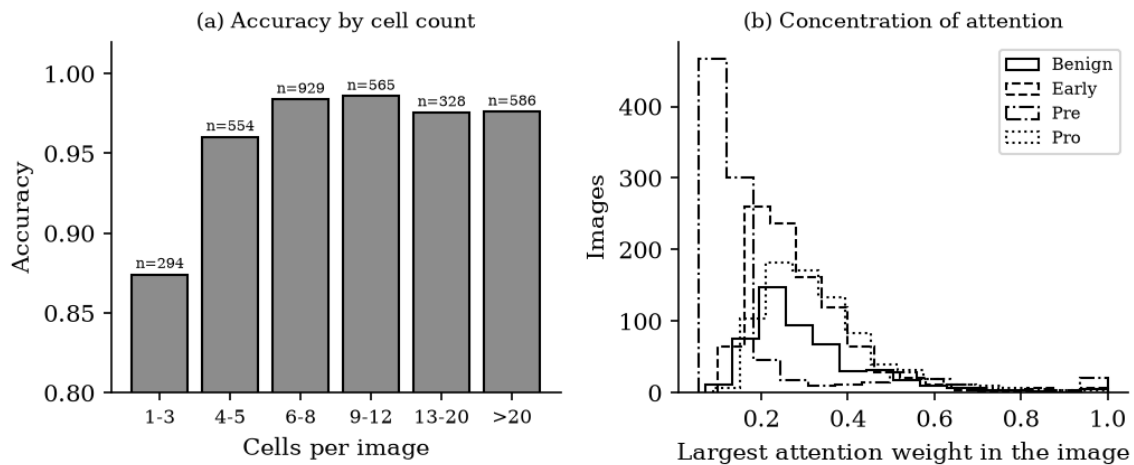


Figure 12. Dependence of the accuracy of the headline model on the number of cells that the pipeline found in the image. (a) Accuracy by number of cells per image (n above each bar is the number of images). (b) Distribution of the largest attention weight in an image, by true class.

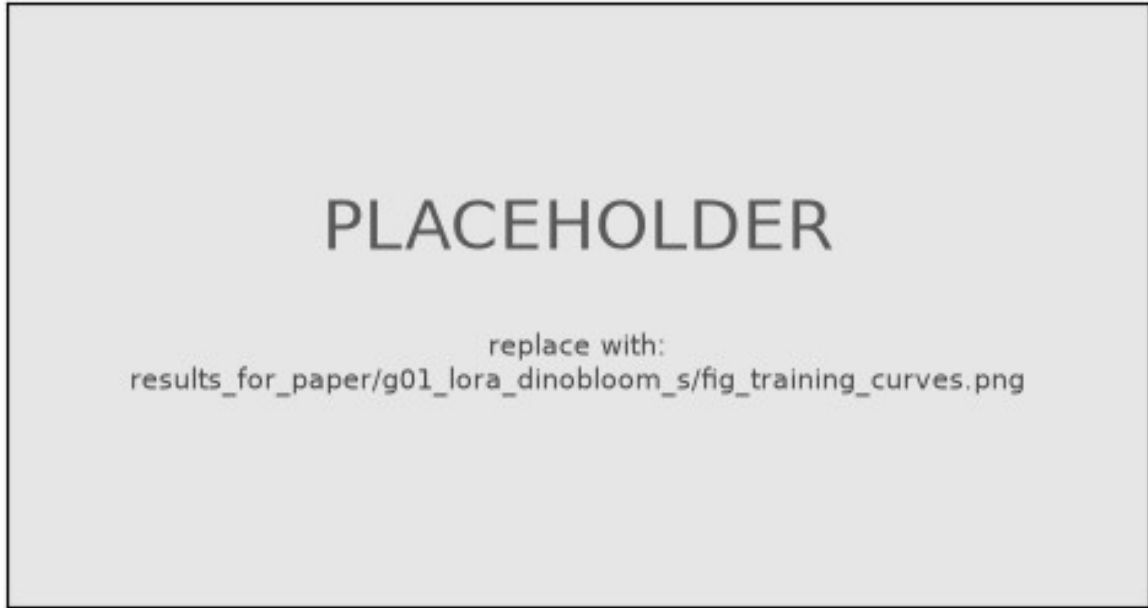


Figure 13. **[PLACEHOLDER - replace before submission]** Training curves of the low-rank adaptation of DinoBloom-S (run g01). Expected content: for each fold, training loss and validation macro-F1 against epoch, with the epoch of the best validation score marked and the learning-rate schedule. Purpose: to show whether adaptation converges, overfits or stops early.

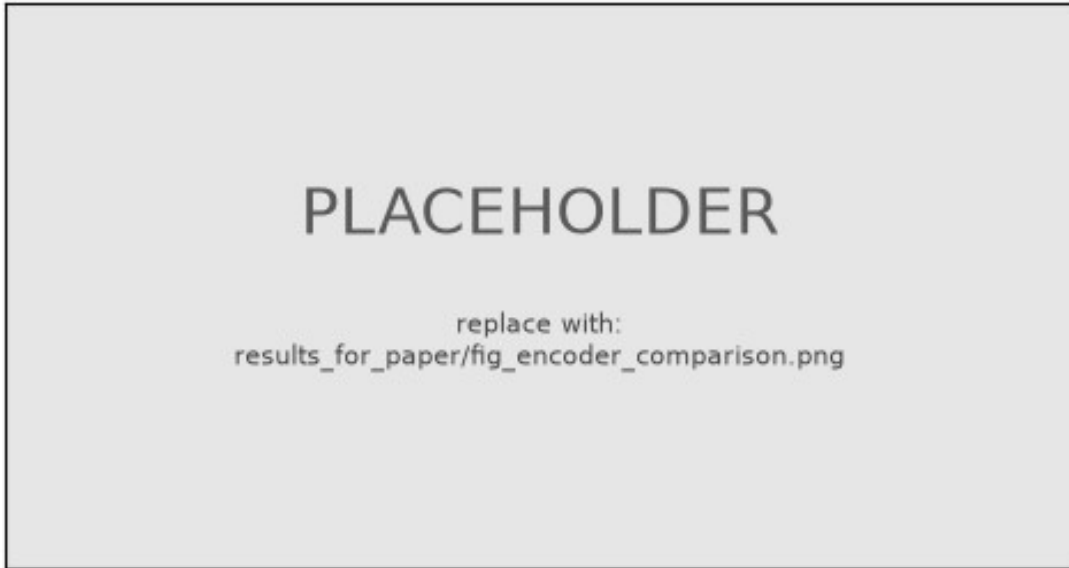


Figure 14. **[PLACEHOLDER - replace before submission]** Balanced accuracy of the fine-tuned configurations. Expected content: one marker per configuration of Table 10 with its 95% cluster-bootstrap interval, sorted by accuracy, with the frozen headline model as a reference line. Purpose: to show which differences exceed the sampling uncertainty.

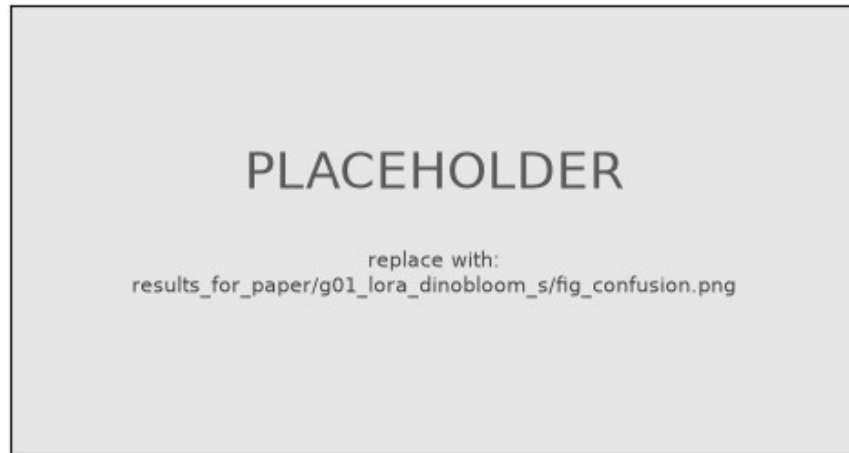


Figure 15. **[PLACEHOLDER - replace before submission]** Confusion matrix of the best fine-tuned configuration. Expected content: pooled out-of-fold confusion matrix, as in Fig. 7a. Purpose: to show whether adaptation changes which classes are confused.



Figure 16. **[PLACEHOLDER - replace before submission]** Attention weights and saliency maps for correctly and incorrectly classified images. Expected content: for each class, example images with the attention weight of each cell and a Grad-CAM map, for one correct and one incorrect prediction, with a sanity check in which the model weights are randomised. Purpose: to show where the model looks and whether it looks at cells or at the periphery.

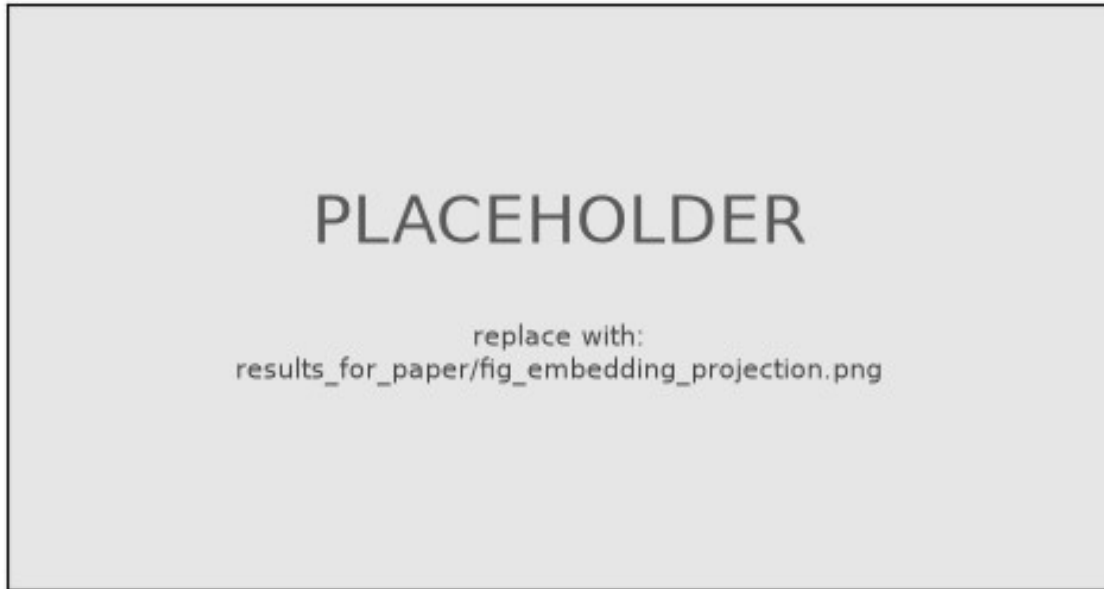


Figure 17. **[PLACEHOLDER - replace before submission]** Projection of the image embeddings. Expected content: a two-dimensional projection of the pooled embeddings of the test images, coloured by class in one panel and by capture-number segment in the other. Purpose: to show whether the embedding separates capture sessions as strongly as it separates classes.

DISCUSSION

A frozen hematology foundation model with an attention head classified the four classes with a balanced accuracy of 0.968 under a protocol that separated capture sessions with an embargo. Three observations qualify this number, and I regard them as the main findings of the study.

First, the dataset contains strong acquisition shortcuts. The mean background colour of an image predicted the class with a balanced accuracy of 0.823, and a model that was allowed to see the background performed better than one that saw only the cells (Table 5). This agrees with the reports of shortcut learning in other medical imaging tasks (DeGrave et al. 2021; Geirhos et al. 2020). The most likely explanation is that the classes were imaged in separate sessions with different illumination and staining; I could not verify this because the dataset documentation does not give per-image acquisition metadata. The class-batch confound means that no analysis of this dataset alone can separate the morphology of the cells from the conditions of the session. Neutralising the background removed the strongest cue, but the colour of the cells remained informative about the class (Table 3), as it would be if the stain differed between sessions and also if the diseases differ in cytoplasmic staining. The accuracy reported here should therefore be read as an upper bound for new patients and new laboratories.

Second, random folds overestimate accuracy, and the effect operates through proximity in capture order. The overestimate was small for the foundation model (1.0 percentage points) and larger for hand-made features (3.0). It is tempting to conclude that the foundation-model features are robust to leakage; a more careful reading is that they are so strong that even a training set from distant sessions remains sufficient (at a gap of 225 capture numbers the accuracy was still 0.932), which is itself compatible with a class-specific session signature that is

stable across the whole class. Distance in capture order is only a proxy for patient identity; without true patient labels I cannot exclude that images of the same patient were present on both sides of every fold.

Third, the preprocessing is useful but it is not what limits accuracy here. The shortcut-neutralised input reduced accuracy relative to the background-visible input (0.959 versus 0.978), which is the expected direction when a shortcut is removed. Hand-made morphological features reached 0.825 and the encoder features 0.959, so the foundation model captured information that my features did not. The nucleus step failed for 92.4% of the cells, which limits what hand-made morphology can add, and the segmentation was not validated against ground-truth masks.

Limitations

The study has limits that the data impose and I could not remove. There are no patient identifiers, and the 'session groups' are a proxy. All classes appear to come from different sessions, so classification accuracy cannot be separated from session identification. There is one dataset and no external test set. Class labels were taken as given, and I did not review them. The DinoBloom training collection may include this dataset or related datasets from the same source; I could not verify this, and if it does, the foundation-model results are optimistic for a further reason. The purge and gap analyses are single runs without intervals. The headline model was trained on a CPU; fine-tuning of the encoder and a comparison of encoders on a GPU was prepared and tested in code but its results are not part of this paper. Calibration was fitted on a validation stretch and assessed on the test stretch of the same dataset, and has not been validated on other data.

Implications and future work

Three practices follow from the results and would improve work on this and similar datasets. Report evaluation with session-aware or patient-aware folds and an embargo whenever capture order is available. Report a shortcut audit - at minimum the accuracy obtained from the background alone - together with any headline accuracy. Validate on an external dataset acquired in a different laboratory. The natural next experiments are an evaluation on an external dataset of leukemic and normal cells such as C-NMC (Mourya et al. 2019) or the Raabin-WBC data (Kouzehkanan et al. 2022), and fine-tuning of the foundation-model encoder under the same folds.

In conclusion, a frozen foundation model separated the four classes of this dataset with a balanced accuracy of 0.968 under a session-aware protocol, but the dataset contains acquisition shortcuts that by themselves predicted the class with a balanced accuracy of 0.823. The accuracy is best interpreted as an upper bound for deployment, and the evaluation design, not the model, is the main determinant of how far such a number can be trusted.

Interpretation of the headline result

The balanced accuracy of 0.968 with an interval of 0.955-0.978 is a measurement of how well four classes of this dataset can be separated by a frozen encoder and a small head when the test images come from capture ranges that are not adjacent to the training images. It is not an estimate of accuracy for patients in general. The interval is the sampling uncertainty over the 90 groups, and it does not include the uncertainty that comes from the unknown relation between groups and patients, from the confounding of class and session, or from the choice of the dataset. Where this paper uses the word accuracy without qualification, it refers to this in-dataset quantity.

Two features of the result are worth stating. First, the frozen encoder reached a high accuracy without any adaptation to the dataset, with a head that has few trainable parameters; the folds that the model fits are therefore unlikely to be overfitted in the usual sense, and the lack of overfitting is consistent with the stability of the per-fold results (standard deviation 0.019). Second, the high accuracy extends to the hand-made features only partially (0.825), which shows that the encoder contains information beyond shape, texture and curvature as I measured them. What this information is cannot be determined from the present experiments. It may include fine nuclear and cytoplasmic texture that is relevant to the disease, and it may include colour and focus characteristics of the sessions.

Relation to other work

Studies that report evaluation on public leukemia datasets typically use random splits of images and report accuracy; the protocol of the present study is stricter, so its numbers are not comparable with theirs and should not be ranked against them. The concern that random splits inflate results is shared by the recent benchmark of Albzour (2026), and the broader literature on leakage (Kapoor and Narayanan 2023; Varoquaux and Cheplygina 2022) describes the same failure in other fields. The contribution of the present analysis is a measurement for this dataset: the size of the overestimate from random block folds, and the proximity mechanism behind it, are quantified instead of assumed.

Consequences for users of the dataset

The finding that the background alone predicts the class has practical consequences. A classifier trained on the original images and evaluated on a random split can reach a high accuracy by recognising the session; such a classifier is likely to fail on images from a laboratory with different staining or imaging, and, worse, it would give no warning. I therefore recommend

that studies using this dataset (1) report accuracy under a session-aware protocol, (2) report the accuracy that a classifier reaches from the background alone, (3) mask or neutralise the background if the goal is to measure cell morphology, and (4) test on an external dataset. The code released with this paper implements the first three, and the fourth requires data that were not part of this study.

Threats to validity

Internal validity. The groups are defined by capture number, which I showed to be informative about similarity, but which may not coincide with patients. If images of a single patient are spread over several capture ranges of the same class, leakage remains even in the contiguous design. The embargo reduces but cannot remove this possibility. The hyperparameters of the head were fixed before the evaluation and were not tuned on the test folds; early stopping and temperature scaling used the validation stretch only. The purge and gap analyses are single runs per setting and the differences they show are large relative to the differences between the random controls, but I did not quantify their sampling variation.

Construct validity. The labels correspond to stages of precursor B-cell leukemia as defined by the dataset authors, and the benign class consists of hematogones. A classifier that separates these classes does not necessarily detect leukemia in a general sample, which contains many other cell types and abnormalities. The four-class task is also a coarse description of a continuum.

External validity. One dataset from one laboratory was used. The acquisition cues identified here may not exist in other datasets, and the accuracy obtained for a different dataset may differ in either direction. I did not test the models on any external data.

Statistical validity. Group-level bootstrap intervals assume that groups are independent and exchangeable, and with 90 groups the intervals are themselves uncertain. The Holm correction was applied within each table, not across all analyses of the paper. Single-seed deterministic comparisons (the head with a fixed seed) do not reflect the variation of training with different initialisations.

Ethical and clinical considerations

The images are from a public dataset released for research. I did not link them to any identifying information and I did not attempt to. The software is a research prototype, not a medical device, and nothing in this paper supports its use for diagnosis. A tool intended for clinical use would need prospective validation on the intended population, evaluation of subgroup performance and regulatory approval, none of which is addressed here.

Expected effect of fine-tuning

Fine-tuning changes the question of the study because it lets the model adapt to the dataset. Two outcomes are possible and they would be informative in different ways. If adaptation of the encoder improves the accuracy under the session-aware protocol, the frozen features were limiting and the dataset contains additional morphological information. If it improves the accuracy mainly when the background is visible (g08) and not when it is neutralised (g01), the gain would be attributable to the shortcut. The experiments were designed so that these two readings can be told apart.

[[GPU: after the runs finish, replace this section by a discussion of which of the two readings the results support; cite Table 10 and Fig. 14. Delete this paragraph if no run is performed.]]

Future work

The most valuable next step is evaluation on an external dataset acquired in a different laboratory, with the same preprocessing and without any adaptation to it. Public datasets with labelled leukemic and normal cells exist, and some of them provide patient identifiers that would also allow a validation of the capture-order proxy. A second step is the segmentation of nuclei, which failed for most cells here; a segmentation network trained on annotated nuclei would allow reliable nuclear measures. A third step is a prospective study of how the confidence of the classifier relates to the disagreement between observers. Finally, an analysis of the encoder with controlled perturbations of colour and background would complement the correlational audit by a causal one.

Conclusion

A reproducible, tested pipeline and a leakage-aware evaluation show that a frozen hematology foundation model separates the four classes of the studied dataset with a balanced accuracy of 0.968, that a large part of the class information is accessible without any cell, and that random block evaluation overestimates accuracy through proximity in capture order. The numbers reported here are an upper bound for new patients and laboratories. The practical message for the field is that the evaluation protocol and a shortcut audit must be part of every report of accuracy on datasets that lack patient identifiers.

ACKNOWLEDGMENTS

ACKNOWLEDGMENTS - FILL IN (advisor, institution, compute providers, dataset authors)

LITERATURE CITED

- Adebayo J, Gilmer J, Muelly M, Goodfellow I, Hardt M, Kim B. 2018. Sanity checks for saliency maps. *Adv Neural Inf Process Syst* 31:9505-9515.
- Albzour N. 2026. A leakage-aware comparative benchmark of machine learning, deep learning, and transformer models for reliable leukemia detection. *arXiv:2606.24944* (preprint).

- Arber DA, Orazi A, Hasserjian R, Thiele J, Borowitz MJ, Le Beau MM, Bloomfield CD, Cazzola M, Vardiman JW. 2016. The 2016 revision to the World Health Organization classification of myeloid neoplasms and acute leukemia. *Blood* 127(20):2391-2405.
- Aria M, Ghaderzadeh M, Bashash D, Abolghasemi H, Asadi F, Hosseini A. 2021. Acute lymphoblastic leukemia (ALL) image dataset. Kaggle. doi:10.34740/KAGGLE/DSV/2175623.
- Bradski G. 2000. The OpenCV library. *Dr Dobbs J Softw Tools* 25(11):120-125.
- Caron M, Touvron H, Misra I, Jegou H, Mairal J, Bojanowski P, Joulin A. 2021. Emerging properties in self-supervised vision transformers. In: *Proc IEEE/CVF Int Conf Comput Vis*. p 9650-9660.
- DeGrave AJ, Janizek JD, Lee SI. 2021. AI for radiographic COVID-19 detection selects shortcuts over signal. *Nat Mach Intell* 3:610-619.
- Dosovitskiy A, Beyer L, Kolesnikov A, Weissenborn D, Zhai X, Unterthiner T, Dehghani M, Minderer M, Heigold G, Gelly S, Uszkoreit J, Houlsby N. 2021. An image is worth 16x16 words: transformers for image recognition at scale. In: *Int Conf Learn Represent*.
- Efron B. 1979. Bootstrap methods: another look at the jackknife. *Ann Stat* 7(1):1-26.
- Frangi AF, Niessen WJ, Vincken KL, Viergever MA. 1998. Multiscale vessel enhancement filtering. In: *Medical Image Computing and Computer-Assisted Intervention. Lecture Notes in Computer Science* vol 1496. Berlin: Springer. p 130-137.
- Geirhos R, Jacobsen JH, Michaelis C, Zemel R, Brendel W, Bethge M, Wichmann FA. 2020. Shortcut learning in deep neural networks. *Nat Mach Intell* 2:665-673.
- Ghaderzadeh M, Aria M, Hosseini A, Asadi F, Bashash D, Abolghasemi H. 2022. A fast and efficient CNN model for B-ALL diagnosis and its subtypes classification using peripheral blood smear images. *Int J Intell Syst* 37(8):5113-5133.
- Guo C, Pleiss G, Sun Y, Weinberger KQ. 2017. On calibration of modern neural networks. *Proc Mach Learn Res* 70:1321-1330.
- Haralick RM, Shanmugam K, Dinstein I. 1973. Textural features for image classification. *IEEE Trans Syst Man Cybern* 3(6):610-621.
- He K, Zhang X, Ren S, Sun J. 2016. Deep residual learning for image recognition. In: *Proc IEEE Conf Comput Vis Pattern Recognit*. p 770-778.
- Holm S. 1979. A simple sequentially rejective multiple test procedure. *Scand J Stat* 6(2):65-70.
- Hu EJ, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, Wang L, Chen W. 2022. LoRA: low-rank adaptation of large language models. In: *Int Conf Learn Represent*.
- Hunger SP, Mullighan CG. 2015. Acute lymphoblastic leukemia in children. *N Engl J Med* 373(16):1541-1552.
- Ilse M, Tomczak JM, Welling M. 2018. Attention-based deep multiple instance learning. *Proc Mach Learn Res* 80:2127-2136.
- Kapoor S, Narayanan A. 2023. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* 4(9):100804.

- Kingma DP, Ba J. 2015. Adam: a method for stochastic optimization. In: Int Conf Learn Represent.
- Koch V, Wagner SJ, Kazeminia S, Sancar E, Hehr M, Schnabel JA, Peng T, Marr C. 2024. DinoBloom: a foundation model for generalizable cell embeddings in hematology. In: Medical Image Computing and Computer-Assisted Intervention. Cham: Springer. arXiv:2404.05022.
- Kohavi R. 1995. A study of cross-validation and bootstrap for accuracy estimation and model selection. In: Proc 14th Int Joint Conf Artif Intell. p 1137-1143.
- Kornblith S, Shlens J, Le QV. 2019. Do better ImageNet models transfer better? In: Proc IEEE Conf Comput Vis Pattern Recognit. p 2661-2671.
- Kouzehkanan ZM, Saghari S, Tavakoli E, Rostami P, Abaszadeh M, Mirzadeh F, Satlsar ES, Gheidishahran M, Gorgi F, Mohammadi S, Hosseini R. 2022. A large dataset of white blood cells containing cell locations and types, along with segmented nuclei and cytoplasm. Sci Rep 12:1123.
- Krizhevsky A, Sutskever I, Hinton GE. 2012. ImageNet classification with deep convolutional neural networks. Adv Neural Inf Process Syst 25:1097-1105.
- Labati RD, Piuri V, Scotti F. 2011. All-IDB: the acute lymphoblastic leukemia image database for image processing. In: Proc 18th IEEE Int Conf Image Process. p 2045-2048.
- Loshchilov I, Hutter F. 2019. Decoupled weight decay regularization. In: Int Conf Learn Represent.
- Macenko M, Niethammer M, Marron JS, Borland D, Woosley JT, Guan X, Schmitt C, Thomas NE. 2009. A method for normalizing histology slides for quantitative analysis. In: Proc IEEE Int Symp Biomed Imaging. p 1107-1110.
- Matek C, Schwarz S, Spiekermann K, Marr C. 2019. Human-level recognition of blast cells in acute myeloid leukaemia with convolutional neural networks. Nat Mach Intell 1:538-544.
- Meyer F, Beucher S. 1990. Morphological segmentation. J Vis Commun Image Represent 1(1):21-46.
- Mourya S, Kant S, Kumar P, Gupta A, Gupta R. 2019. ALL challenge dataset of ISBI 2019 (C-NMC 2019). The Cancer Imaging Archive. doi:10.7937/tcia.2019.dc64i46r.
- Nadeau C, Bengio Y. 2003. Inference for the generalization error. Mach Learn 52(3):239-281.
- Oquab M, Darcet T, Moutakanni T, Vo H, Szafraniec M, Khalidov V, Fernandez P, Haziza D, Massa F, El-Nouby A, Assran M, Ballas N, Galuba W, Howes R, Huang PY, Li SW, Misra I, Rabbat M, Sharma V, Synnaeve G, Xu H, Jegou H, Mairal J, Labatut P, Joulin A, Bojanowski P. 2024. DINOv2: learning robust visual features without supervision. Trans Mach Learn Res.
- Otsu N. 1979. A threshold selection method from gray-level histograms. IEEE Trans Syst Man Cybern 9(1):62-66.
- Pan SJ, Yang Q. 2010. A survey on transfer learning. IEEE Trans Knowl Data Eng 22(10):1345-1359.

- Paszke A, Gross S, Massa F, Lerer A, Bradbury J, Chanan G, Killeen T, Lin Z, Gimelshein N, Antiga L, Desmaison A, Kopf A, Yang E, DeVito Z, Raison M, Tejani A, Chilamkurthy S, Steiner B, Fang L, Bai J, Chintala S. 2019. PyTorch: an imperative style, high-performance deep learning library. *Adv Neural Inf Process Syst* 32:8024-8035.
- Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, Blondel M, Prettenhofer P, Weiss R, Dubourg V, Vanderplas J, Passos A, Cournapeau D, Brucher M, Perrot M, Duchesnay E. 2011. Scikit-learn: machine learning in Python. *J Mach Learn Res* 12:2825-2830.
- Perona P, Malik J. 1990. Scale-space and edge detection using anisotropic diffusion. *IEEE Trans Pattern Anal Mach Intell* 12(7):629-639.
- Reinhard E, Adhikhmin M, Gooch B, Shirley P. 2001. Color transfer between images. *IEEE Comput Graph Appl* 21(5):34-41.
- Roberts DR, Bahn V, Ciuti S, Boyce MS, Elith J, Guillera-Arroita G, Hauenstein S, Lahoz-Monfort JJ, Schroder B, Thuiller W, Warton DI, Wintle BA, Hartig F, Dormann CF. 2017. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography* 40(8):913-929.
- Roberts M, Driggs D, Thorpe M, Gilbey J, Yeung M, Ursprung S, Aviles-Rivero AI, Etmann C, McCague C, Beer L, and others. 2021. Common pitfalls and recommendations for using machine learning to detect and prognosticate for COVID-19 using chest radiographs and CT scans. *Nat Mach Intell* 3:199-217.
- Ronneberger O, Fischer P, Brox T. 2015. U-Net: convolutional networks for biomedical image segmentation. In: *Medical Image Computing and Computer-Assisted Intervention*. Cham: Springer. p 234-241.
- Schmidt U, Weigert M, Broaddus C, Myers G. 2018. Cell detection with star-convex polygons. In: *Medical Image Computing and Computer-Assisted Intervention*. Cham: Springer. p 265-273.
- Selvaraju RR, Cogswell M, Das A, Vedantam R, Parikh D, Batra D. 2017. Grad-CAM: visual explanations from deep networks via gradient-based localization. In: *Proc IEEE Int Conf Comput Vis*. p 618-626.
- Sokolova M, Lapalme G. 2009. A systematic analysis of performance measures for classification tasks. *Inf Process Manag* 45(4):427-437.
- Stringer C, Wang T, Michaelos M, Pachitariu M. 2021. Cellpose: a generalist algorithm for cellular segmentation. *Nat Methods* 18:100-106.
- Tan M, Le QV. 2019. EfficientNet: rethinking model scaling for convolutional neural networks. *Proc Mach Learn Res* 97:6105-6114.
- Tellez D, Litjens G, Bandi P, Bulten W, Bokhorst JM, Ciompi F, van der Laak J. 2019. Quantifying the effects of data augmentation and stain color normalization in convolutional neural networks for computational pathology. *Med Image Anal* 58:101544.
- Tomasi C, Manduchi R. 1998. Bilateral filtering for gray and color images. In: *Proc 6th Int Conf Comput Vis*. p 839-846.

- Vahadane A, Peng T, Sethi A, Albarqouni S, Wang L, Baust M, Steiger K, Schlitter AM, Esposito I, Navab N. 2016. Structure-preserving color normalization and sparse stain separation for histological images. *IEEE Trans Med Imaging* 35(8):1962-1971.
- van der Walt S, Schonberger JL, Nunez-Iglesias J, Boulogne F, Warner JD, Yager N, Gouillart E, Yu T. 2014. scikit-image: image processing in Python. *PeerJ* 2:e453.
- Varoquaux G, Cheplygina V. 2022. Machine learning for medical imaging: methodological failures and recommendations for the future. *NPJ Digit Med* 5:48.
- Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I. 2017. Attention is all you need. *Adv Neural Inf Process Syst* 30:5998-6008.
- Wightman R. 2019. PyTorch image models. GitHub repository, <https://github.com/huggingface/pytorch-image-models>.

APPENDIX I. PER-FOLD RESULTS OF THE HEADLINE MODEL

Table A1 lists the results of each of the five folds. Table A2 lists per-class recall in each fold.

These are the unpooled test-stretch results from which the pooled numbers in the main text are computed.

Table A1. Per-fold results of the headline model (frozen DinoBloom-S, attention head).

Fold	Train	Valid.	Test	Bal. acc.	Macro-F1	AUROC	Epochs	Temp.	Time (s)
1	1508	652	652	0.954	0.948	0.996	18	0.38	468
2	1361	651	652	0.943	0.936	0.989	14	0.24	356
3	1511	649	652	0.977	0.979	0.997	12	0.33	327
4	1509	652	651	0.989	0.991	1.000	17	0.48	459
5	1511	652	649	0.976	0.980	0.999	23	0.55	363

Table A2. Per-class recall of each fold (test stretch).

Fold	Benign	Early	Pre	Pro
1	0.980	0.990	0.902	0.944
2	0.990	0.909	0.891	0.981
3	0.950	0.975	0.995	0.988
4	0.960	0.995	1.000	1.000
5	0.920	0.995	1.000	0.988

Table A3. Pooled out-of-fold confusion matrix of the headline model (rows: true class; columns: predicted class).

True \ predicted	Benign	Early	Pre	Pro
Benign	484	9	10	1
Early	14	958	7	6
Pre	10	16	922	15
Pro	3	4	9	788

Table A4. Confusion matrices of the five folds (rows: true class; columns: predicted class).

Fold	True class	Benign	Early	Pre	Pro
1	Benign	99	0	2	0
	Early	1	195	0	1
	Pre	10	3	174	6
	Pro	3	3	3	152
2	Benign	100	1	0	0
	Early	12	179	6	0
	Pre	0	13	172	8
	Pro	0	0	3	158
3	Benign	96	1	4	0
	Early	1	192	1	3
	Pre	0	0	192	1
	Pro	0	1	1	159
4	Benign	97	2	2	0
	Early	0	196	0	1
	Pre	0	0	192	0
	Pro	0	0	0	161
5	Benign	92	5	2	1
	Early	0	196	0	1
	Pre	0	0	192	0
	Pro	0	0	2	158

APPENDIX II. SUPERSEDED RANDOM-BLOCK ABLATION

The ablation of Table 5 was first computed with random-block folds. These results are kept because they show the size of the inflation; they are not used as evidence of performance.

Table A5. Representation ablation with random 37-image blocks (superseded by the contiguous protocol).

Variant	Balanced accuracy	Macro-F1	AUROC
A. clean RGB, isolated cell (reference)	0.969	0.969	0.998
B. clean grayscale, isolated cell (no stain colour)	0.959	0.958	0.995
C. normalised RGB, background visible	0.988	0.988	0.999
D. raw RGB, background visible	0.992	0.992	1.000
E. tabular shape+texture+curvature (no pixels)	0.854	0.848	0.959

APPENDIX III. DEFINITIONS OF THE PER-CELL FEATURES

Each segmented cell yields the following measurements. Image-level aggregates are the mean, standard deviation and maximum over the cells of an image (computed over interior cells unless the image has none). All lengths are in pixels and all areas in square pixels of the 224 x 224 image.

Table A6. Per-cell features computed by the preprocessing package.

Group	Columns	Definition
Shape	cell_area, cell_perimeter, cell_eq_diameter	Area, perimeter and equivalent-circle diameter of the cell mask.
Shape	cell_major_axis, cell_minor_axis, cell_aspect_ratio, cell_eccentricity	Ellipse-fit axes of the mask and derived ratios.
Shape	cell_solidity, cell_extent, cell_circularity	Area over convex-hull area; area over bounding-box area; $4 \pi \text{ area} / \text{perimeter}^2$.
Position	centroid_y, centroid_x, touches_border	Centroid and whether the mask touches the image border.
Colour	cell_L_mean/std, cell_a_mean/std, cell_b_mean/std	Mean and SD of the CIE Lab channels inside the cell (stain-normalised image).
Nucleus	nucleus_found, nucleus_contrast	Whether the nucleus step succeeded, and the contrast used to accept it.
Nucleus	nucleus_area ... nucleus_circularity	The same shape measurements for the nucleus mask (missing when no nucleus was found).
Nucleus	nc_ratio	Nucleus area divided by cell area.
Nucleus	cytoplasm_L/a/b_mean, nuc_cyto_L_contrast	Lab means of the cytoplasm (cell minus nucleus) and nucleus-cytoplasm lightness contrast.
Texture	cell_glcm_contrast/homogeneity/energy/correlation	Grey-level co-occurrence statistics (Haralick et al. 1973) of the cell, masked and averaged over four directions.
Texture	nucleus_glcm_*, nucleus_entropy, cell_entropy	The same for the nucleus; Shannon entropy of the grey-level histogram.
Response	cell_apc_mean/std/rim_mean	Mean, SD and rim mean of the principal-curvature response layer inside the cell.
Response	cell_log_mean/std/rim_mean	The same for the Laplacian-of-Gaussian layer.

APPENDIX IV. SOFTWARE AND PREPARED EXPERIMENTS

The repository contains two packages. The first (leukemia_pp) implements the preprocessing described in Materials and Methods and exposes the commands run, inspect and audit. The second (leukemia_ml) implements encoder loading, low-rank adaptation, attention pooling, a hybrid branch, training with exact resume, cross-validation, calibration, statistics and Grad-CAM. The test suite contains 189 tests and was run on a CPU.

Eight GPU experiments (configurations g01 to g08) were prepared for fine-tuning: low-rank adaptation of DinoBloom-S (g01), a grayscale variant (g02), the hybrid model with hand-made features (g03), full fine-tuning of ResNet-50 (g04) and EfficientNet-B0 (g05), low-rank adaptation of generic DINOv2-S (g06) and of DinoBloom-B (g07), and a raw image with background (g08). They were tested for correctness on a CPU with a few steps, but their results are not part of this paper.

Headline model configuration (JSON):

```
{
  "name": "cv_dinobloom_s_frozen_mil",
  "data": {
    "source": "clean_rgb",
    "isolate": true,
    "task": "4way",
    "feature_groups": [
      "cell_shape",
      "cell_texture",
      "apc",
      "log"
    ],
    "split_scheme": "contiguous",
    "n_folds": 5,
    "val_offset": 2,
    "embargo": 37
  },
  "model": {
    "encoder": "dinobloom_s",
    "weights_path": "data/weights/DinoBloom-S.pth",
    "input_size": 112,
    "freeze": "frozen",
    "head": "mil",
    "use_features": false
  },
  "train": {
    "epochs": 30,
    "batch_size": 16,
    "lr_head": 0.001,

```

```

    "patience": 8,
    "seed": 0,
    "device": "cpu"
  }
}

```

Preprocessing configuration of the full run (JSON):

```

{
  "data": {
    "max_side": null
  },
  "split": {
    "train": 0.7,
    "val": 0.15,
    "test": 0.15,
    "seed": 42,
    "group_regex": null,
    "sequence_block": 37
  },
  "stain": {
    "enabled": true,
    "ref_sample_size": 20,
    "min_foreground_frac": 0.02,
    "scale_clip": [
      0.25,
      4.0
    ]
  },
  "denoise": {
    "method": "bilateral",
    "bilateral_d": 9,
    "bilateral_sigma_color": 75.0,
    "bilateral_sigma_space": 75.0,
    "nlm_h": 5.0,
    "nlm_h_color": 5.0
  },
  "response": {
    "apc_sigma": 2.5,
    "apc_alpha": 0.6,
    "log_sigma": 4.0,
    "apc_gain": 12.0,
    "log_gain": 5.0,
    "canny_low_ratio": 0.5
  },
  "hds": {
    "enabled": false,
    "iterations": 30,
    "dt": 0.2,
    "lam": 0.3,
    "h": 0.05,
    "beta": 0.05,
    "hybrid_weight": 0.5,
    "tol": 0.0001
  },
  "beemd": {
    "enabled": false,
    "n_imfs": 4,
    "ensemble": 8,
    "noise_std": 0.2,
    "max_sifts": 8,
    "sd_tol": 0.2,
    "seed": 0,
  }
}

```

```

    "min_scale_px": 3.0,
    "max_scale_px": 48.0,
    "min_energy_frac": 0.01
  },
  "neutralise": {
    "enabled": true,
    "dilate_px": 2,
    "fill_l": 92.0,
    "white_balance": true,
    "gain_clip": [
      0.7,
      1.3
    ],
    "min_background_px": 200
  },
  "segmentation": {
    "smooth_sigma": 1.0,
    "a_floor": 8.0,
    "min_cell_area": 120,
    "open_radius": 1,
    "split_h": 1.5,
    "split_smooth": 1.0,
    "nucleus_smooth_sigma": 1.0,
    "nucleus_rim_erode": 2,
    "nucleus_min_contrast": 8.0,
    "nucleus_min_area": 30,
    "nucleus_min_cell_frac": 0.05,
    "nucleus_keep_frac": 0.2
  },
  "features": {
    "glcm_levels": 32,
    "glcm_distances": [
      1,
      2,
      3
    ],
    "glcm_angles_deg": [
      0,
      45,
      90,
      135
    ],
    "min_texture_pixels": 30
  },
  "qc": {
    "min_sharpness": 20.0,
    "min_brightness": 40.0,
    "max_wbc_fraction": 0.5,
    "max_border_cell_frac": 0.5
  },
  "class_map": {
    "Benign": [
      0,
      0
    ],
    "Early": [
      1,
      1
    ],
    "Pre": [
      1,
      2
    ]
  },

```

```
"Pro": [  
  1,  
  3  
]  
},  
"save_previews": true  
}
```

APPENDIX V. REPRODUCING THE RESULTS

The commands below reproduce the preprocessing, the audit, the cross-validation and this manuscript from the images of the public dataset (a Kaggle account is needed to download them).

```
pip install -e .
leukemia-pp run --input data/dataset --output data/full_run_v2 --config
configs/full_dataset.json
leukemia-pp audit --run data/full_run_v2
leukemia-ml cv --config configs/ml/cv_dinobloom_s_frozen_mil.json --out
runs/cv_dinobloom_s
leukemia-ml ablate --spec configs/ml/ablation_sources.json
python scripts/analysis_dataset.py; python scripts/analysis_session.py
python scripts/analysis_leakage.py; python scripts/benchmark_hds.py
python scripts/make_figures.py
python paper/build_paper.py && node paper/render_docx.js paper/content.json
paper/paper.docx
```

The weights of DinoBloom-S are available from Zenodo (record 10908163) under a Creative Commons Attribution licence.

APPENDIX VI. IMAGES MISCLASSIFIED BY THE HEADLINE MODEL

Table A7 lists the 104 of 3256 images that the headline model classified incorrectly in the out-of-fold evaluation, with the fold in which each was tested, the true and predicted class, the confidence of the prediction and the number of cells found in the image. The list is sorted by file name.

Table A7. Misclassified images (pooled out-of-fold predictions).

Image	Fold	True class	Predicted	Confidence	Cells
WBC-Benign-047	1	Benign	Pre	0.74	7
WBC-Benign-075	1	Benign	Pre	0.53	8
WBC-Benign-116	2	Benign	Early	0.65	4
WBC-Benign-211	3	Benign	Pre	1.00	22
WBC-Benign-242	3	Benign	Pre	1.00	8
WBC-Benign-280	3	Benign	Pre	0.69	2
WBC-Benign-292	3	Benign	Early	0.98	5
WBC-Benign-298	3	Benign	Pre	0.82	2
WBC-Benign-315	4	Benign	Pre	0.64	8
WBC-Benign-339	4	Benign	Early	0.93	3
WBC-Benign-342	4	Benign	Early	0.53	4
WBC-Benign-390	4	Benign	Pre	0.69	3
WBC-Benign-413	5	Benign	Early	0.61	14
WBC-Benign-416	5	Benign	Early	0.99	9
WBC-Benign-421	5	Benign	Pro	0.84	11
WBC-Benign-428	5	Benign	Pre	0.73	13
WBC-Benign-439	5	Benign	Early	0.90	7
WBC-Benign-458	5	Benign	Early	0.83	4
WBC-Benign-472	5	Benign	Early	0.73	10
WBC-Benign-484	5	Benign	Pre	0.65	17
WBC-Malignant-Early-142	1	Early	Pro	0.53	4
WBC-Malignant-Early-168	1	Early	Benign	0.71	7
WBC-Malignant-Early-210	2	Early	Benign	0.42	8
WBC-Malignant-Early-211	2	Early	Benign	0.42	7
WBC-Malignant-Early-228	2	Early	Benign	0.42	4
WBC-Malignant-Early-229	2	Early	Pre	0.40	4
WBC-Malignant-Early-235	2	Early	Benign	1.00	4
WBC-Malignant-Early-236	2	Early	Benign	0.57	7
WBC-Malignant-Early-246	2	Early	Benign	0.84	3
WBC-Malignant-Early-248	2	Early	Benign	0.94	6
WBC-Malignant-Early-267	2	Early	Benign	0.99	11
WBC-Malignant-Early-276	2	Early	Benign	0.77	6
WBC-Malignant-Early-277	2	Early	Pre	0.68	8
WBC-Malignant-Early-282	2	Early	Benign	0.62	6
WBC-Malignant-Early-290	2	Early	Benign	0.92	30
WBC-Malignant-Early-311	2	Early	Pre	0.87	5
WBC-Malignant-Early-312	2	Early	Pre	0.90	4
WBC-Malignant-Early-347	2	Early	Pre	1.00	4
WBC-Malignant-Early-369	2	Early	Pre	0.99	5

Image	Fold	True class	Predicted	Confidence	Cells
WBC-Malignant-Early-385	2	Early	Benign	0.99	5
WBC-Malignant-Early-402	3	Early	Pro	0.65	29
WBC-Malignant-Early-504	3	Early	Pro	0.98	17
WBC-Malignant-Early-540	3	Early	Benign	0.99	2
WBC-Malignant-Early-571	3	Early	Pro	0.66	2
WBC-Malignant-Early-583	3	Early	Pre	0.58	8
WBC-Malignant-Early-703	4	Early	Pro	0.63	4
WBC-Malignant-Early-982	5	Early	Pro	0.73	7
WBC-Malignant-Pre-001	1	Pre	Benign	0.86	27
WBC-Malignant-Pre-002	1	Pre	Early	0.76	3
WBC-Malignant-Pre-004	1	Pre	Benign	0.66	23
WBC-Malignant-Pre-007	1	Pre	Pro	0.88	22
WBC-Malignant-Pre-010	1	Pre	Benign	0.69	27
WBC-Malignant-Pre-013	1	Pre	Benign	0.99	31
WBC-Malignant-Pre-015	1	Pre	Pro	0.99	1
WBC-Malignant-Pre-016	1	Pre	Pro	0.41	17
WBC-Malignant-Pre-019	1	Pre	Early	0.38	21
WBC-Malignant-Pre-022	1	Pre	Benign	0.96	32
WBC-Malignant-Pre-025	1	Pre	Pro	0.35	27
WBC-Malignant-Pre-028	1	Pre	Benign	0.78	21
WBC-Malignant-Pre-032	1	Pre	Benign	0.68	5
WBC-Malignant-Pre-084	1	Pre	Early	0.53	11
WBC-Malignant-Pre-098	1	Pre	Benign	0.40	11
WBC-Malignant-Pre-118	1	Pre	Pro	0.89	3
WBC-Malignant-Pre-123	1	Pre	Benign	0.84	2
WBC-Malignant-Pre-139	1	Pre	Pro	0.51	11
WBC-Malignant-Pre-169	1	Pre	Benign	1.00	14
WBC-Malignant-Pre-197	2	Pre	Early	0.94	1
WBC-Malignant-Pre-199	2	Pre	Early	0.60	2
WBC-Malignant-Pre-201	2	Pre	Pro	0.97	1
WBC-Malignant-Pre-203	2	Pre	Pro	0.47	3
WBC-Malignant-Pre-209	2	Pre	Early	0.99	2
WBC-Malignant-Pre-213	2	Pre	Early	1.00	3
WBC-Malignant-Pre-215	2	Pre	Early	1.00	2
WBC-Malignant-Pre-217	2	Pre	Early	0.63	3
WBC-Malignant-Pre-219	2	Pre	Early	0.98	2
WBC-Malignant-Pre-221	2	Pre	Early	0.74	2
WBC-Malignant-Pre-223	2	Pre	Pro	0.91	4
WBC-Malignant-Pre-225	2	Pre	Pro	1.00	2
WBC-Malignant-Pre-227	2	Pre	Pro	1.00	2

Image	Fold	True class	Predicted	Confidence	Cells
WBC-Malignant-Pre-229	2	Pre	Pro	0.80	3
WBC-Malignant-Pre-233	2	Pre	Pro	1.00	3
WBC-Malignant-Pre-235	2	Pre	Early	1.00	3
WBC-Malignant-Pre-237	2	Pre	Pro	0.94	2
WBC-Malignant-Pre-243	2	Pre	Early	0.96	3
WBC-Malignant-Pre-245	2	Pre	Early	1.00	3
WBC-Malignant-Pre-255	2	Pre	Early	0.85	2
WBC-Malignant-Pre-273	2	Pre	Early	0.94	3
WBC-Malignant-Pre-412	3	Pre	Pro	0.78	23
WBC-Malignant-Pro-011	1	Pro	Benign	0.73	5
WBC-Malignant-Pro-012	1	Pro	Benign	0.73	5
WBC-Malignant-Pro-013	1	Pro	Early	0.51	4
WBC-Malignant-Pro-024	1	Pro	Early	0.58	2
WBC-Malignant-Pro-033	1	Pro	Pre	0.96	3
WBC-Malignant-Pro-040	1	Pro	Pre	0.52	4
WBC-Malignant-Pro-041	1	Pro	Pre	0.74	2
WBC-Malignant-Pro-045	1	Pro	Benign	0.90	16
WBC-Malignant-Pro-156	1	Pro	Early	0.88	4
WBC-Malignant-Pro-175	2	Pro	Pre	0.57	5
WBC-Malignant-Pro-271	2	Pro	Pre	0.82	3
WBC-Malignant-Pro-282	2	Pro	Pre	0.73	3
WBC-Malignant-Pro-343	3	Pro	Pre	0.84	33
WBC-Malignant-Pro-409	3	Pro	Early	0.83	1
WBC-Malignant-Pro-800	5	Pro	Pre	0.87	12
WBC-Malignant-Pro-801	5	Pro	Pre	0.53	15

APPENDIX VII. TRAINING LOGS OF THE FIVE FOLDS

Tables A8 to A12 list the training loss and the validation scores at each epoch of each fold of the headline model.

Table A8. Epoch log of fold 1 (train loss, validation macro-F1, validation balanced accuracy, seconds per epoch).

Epoch	Train loss	Val macro-F1	Val bal. acc.	Seconds
1	0.9723	0.925	0.924	17.6
2	0.4968	0.973	0.974	8.8
3	0.4516	0.972	0.969	24.9
4	0.4303	0.976	0.979	34.7
5	0.4104	0.973	0.975	2.7
6	0.4080	0.982	0.982	36.2
7	0.3977	0.985	0.983	25.1
8	0.3904	0.984	0.983	13.5
9	0.3859	0.986	0.984	11.8
10	0.3838	0.988	0.987	38.3
11	0.3821	0.978	0.976	23.8
12	0.3802	0.986	0.984	37.1
13	0.3797	0.983	0.981	21.8
14	0.3780	0.976	0.972	31.8
15	0.3777	0.981	0.978	12.1
16	0.3769	0.987	0.985	22.5
17	0.3761	0.985	0.983	12.4
18	0.3759	0.985	0.984	35.1

Table A9. Epoch log of fold 2 (train loss, validation macro-F1, validation balanced accuracy, seconds per epoch).

Epoch	Train loss	Val macro-F1	Val bal. acc.	Seconds
1	1.0220	0.948	0.951	20.4
2	0.5244	0.983	0.983	13.0
3	0.4780	0.979	0.980	20.1
4	0.4443	0.985	0.983	14.1
5	0.4255	0.991	0.992	28.2
6	0.4135	0.993	0.993	19.7
7	0.4034	0.991	0.990	33.7
8	0.3989	0.991	0.990	19.8
9	0.3937	0.987	0.986	16.7
10	0.3913	0.989	0.988	22.3
11	0.3900	0.986	0.985	7.5
12	0.3886	0.989	0.987	31.2
13	0.3865	0.988	0.986	32.2
14	0.3867	0.987	0.984	30.7

Table A10. Epoch log of fold 3 (train loss, validation macro-F1, validation balanced accuracy, seconds per epoch).

Epoch	Train loss	Val macro-F1	Val bal. acc.	Seconds
1	0.9974	0.950	0.952	31.4
2	0.5240	0.947	0.944	11.1
3	0.4645	0.968	0.965	36.4
4	0.4405	0.983	0.983	19.3
5	0.4210	0.957	0.957	16.8
6	0.4050	0.970	0.965	5.8
7	0.3988	0.939	0.927	19.0
8	0.3939	0.968	0.962	10.3
9	0.3905	0.972	0.966	34.3
10	0.3859	0.972	0.966	42.7
11	0.3831	0.958	0.950	19.0
12	0.3811	0.975	0.970	35.6

Table A11. Epoch log of fold 4 (train loss, validation macro-F1, validation balanced accuracy, seconds per epoch).

Epoch	Train loss	Val macro-F1	Val bal. acc.	Seconds
1	0.9849	0.831	0.842	19.6
2	0.5063	0.906	0.913	28.4
3	0.4544	0.919	0.927	26.9
4	0.4385	0.938	0.946	26.1
5	0.4136	0.941	0.944	10.6
6	0.4014	0.938	0.939	24.4
7	0.3972	0.941	0.942	30.9
8	0.3917	0.949	0.952	39.9
9	0.3865	0.958	0.960	19.2
10	0.3859	0.953	0.956	11.6
11	0.3828	0.949	0.950	16.3
12	0.3812	0.946	0.948	35.9
13	0.3804	0.949	0.954	23.0
14	0.3789	0.955	0.957	23.9
15	0.3783	0.951	0.954	25.6
16	0.3776	0.953	0.954	18.6
17	0.3772	0.942	0.942	25.0

Table A12. Epoch log of fold 5 (train loss, validation macro-F1, validation balanced accuracy, seconds per epoch).

Epoch	Train loss	Val macro-F1	Val bal. acc.	Seconds
1	0.9901	0.909	0.912	26.4
2	0.5207	0.942	0.940	34.6
3	0.4523	0.913	0.918	36.6
4	0.4355	0.953	0.956	18.4
5	0.4212	0.957	0.960	21.7
6	0.4074	0.952	0.956	24.7
7	0.4012	0.950	0.950	30.5
8	0.3915	0.956	0.958	22.7
9	0.3866	0.955	0.954	38.6
10	0.3851	0.953	0.957	30.2
11	0.3831	0.958	0.961	24.6
12	0.3814	0.964	0.964	1.1
13	0.3812	0.959	0.960	1.1
14	0.3788	0.963	0.960	1.0
15	0.3777	0.968	0.969	1.0
16	0.3776	0.961	0.961	0.9
17	0.3766	0.964	0.964	1.0
18	0.3762	0.965	0.965	1.0
19	0.3761	0.963	0.963	1.2
20	0.3755	0.966	0.967	1.1
21	0.3755	0.958	0.958	1.0
22	0.3753	0.964	0.964	0.9
23	0.3750	0.964	0.963	1.1

APPENDIX VIII. CONFIGURATIONS OF THE FINE-TUNING EXPERIMENTS

Table A13 lists the settings that differ between the eight prepared configurations; all other settings are identical and are given in the configuration listing of Appendix IV and in the files configs/ml/gpu/g01 to g08 of the repository.

Table A13. Settings that differ between the fine-tuning configurations.

Run	Encoder	Adaptation	Input	Isolated	Features	Batch	Encoder learning rate
g01	dinobloom_s	lora	clean_rgb	yes	no	16	0.0002
g02	dinobloom_s	lora	clean_gray	yes	no	16	0.0002
g03	dinobloom_s	lora	clean_rgb	yes	yes	16	0.0002
g04	resnet50.a1_in1k	full	clean_rgb	yes	no	16	0.0001
g05	efficientnet_b0.ra_in1k	full	clean_rgb	yes	no	16	0.0002
g06	dinov2_s	lora	clean_rgb	yes	no	16	0.0002
g07	dinobloom_b	lora	clean_rgb	yes	no	8	0.0002
g08	dinobloom_s	lora	raw	no	no	16	0.0002

Adaptation: low-rank (lora) or full. The learning rate shown is that of the adapter weights for low-rank adaptation and of the encoder for full fine-tuning; the head uses 0.001 in all runs.

Table A14. Settings shared by all fine-tuning configurations.

Setting	Value
Task	four classes
Folds	5 contiguous, validation offset 2, embargo 37
Cells per bag (train / evaluation)	16 / 64
Crop (stored / network input)	128 x 128 / 112 x 112 pixels
Epochs (maximum) / patience	12 / 4
Head	gated attention, 256 hidden units, dropout 0.2
Optimiser	AdamW, weight decay 0.05, warm-up 10% then cosine
Loss	cross-entropy, label smoothing 0.1, inverse-square-root class weights
Gradient clipping	norm 1
Mixed precision	bfloat16 where supported, otherwise float16
Low-rank adaptation	rank 8, scaling 16, attention qkv and projection layers
Seed	0

APPENDIX IX. ALGORITHMS IN PSEUDOCODE

The three procedures that determine the evaluation are given in pseudocode. They correspond to the functions `contiguous_folds`, `purge_train`, `cluster_bootstrap` and `holm` of the code.

Algorithm 1. Contiguous session-aware folds with embargo.

```
input: labels y, capture numbers n, number of folds K, embargo e
for each class c:
    sort the images of class c by n; cut the sorted list into K consecutive
    segments S_1..S_K
    assign every image of segment S_k to fold k
for k = 1..K:
    test = images of fold k; train = images of all other folds
    for each class c: remove from train every image of class c with |n_i - n_j| <=
    e for some test image j of class c
    yield (train, test)
```

Algorithm 2. Cluster bootstrap interval for a metric m.

```
input: groups g(i), metric m(indices), number of resamples B
for b = 1..B:
    draw G groups with replacement, G = number of distinct groups
    idx = concatenation of the images of the drawn groups (a group drawn twice
    contributes twice)
    v_b = m(idx)
return the 2.5th and 97.5th percentiles of v_1..v_B, and the standard deviation of
v
```

Algorithm 3. Paired comparison and Holm adjustment.

```
input: metrics m_A and m_B on the same groups, B resamples
for b = 1..B: draw groups as above; d_b = m_A(idx) - m_B(idx)
p = 2 * min( share of d_b <= 0, share of d_b >= 0 ), floored at 1/B
Holm: sort the p values ascending p_(1) <= ... <= p_(M); p'_(r) = max_(s <= r)
min(1, (M - s + 1) * p_(s))
```

APPENDIX X. GLOSSARY OF TERMS AND ABBREVIATIONS

Table A15. Glossary.

Term	Meaning
a, b, L*	Axes of the CIE Lab colour space: green-red, blue-yellow and lightness.
APC	Adaptive principal curvature: a response built from the eigenvalues of the image Hessian.
Attention (MIL)	A learned weighting of the cells of an image that sums to one.
AUROC	Area under the receiver operating characteristic curve.
Balanced accuracy	Mean of the recalls of the classes.
Bag	The set of cells of one image, treated as one training example.
Bootstrap (cluster)	Resampling of whole groups of images with replacement.
ECE	Expected calibration error.
Embargo	Number of capture numbers around a test image from which training images are removed.
Foundation model	A large model pre-trained on broad data and adapted to many tasks.
Frozen encoder	An encoder whose weights are not changed during training.
GLCM	Grey-level co-occurrence matrix, the basis of Haralick texture features.
Group	A set of images assumed to share a session or patient (here consecutive blocks of 37 capture numbers and duplicates).
HDS	Hybrid diffusion-steered denoising (Perona-Malik plus total variation).
LoG	Laplacian of Gaussian.
LoRA	Low-rank adaptation of the weights of a network.
MIL	Multiple-instance learning.
Neutralisation	Replacement of everything outside the cells by a constant colour.
PBS	Peripheral blood smear.
Purge	Removal of training images that lie close to the test images.
Shortcut	A feature that predicts the label in the dataset without being causally related to it.
Temperature scaling	Division of the logits by a scalar fitted on validation data to calibrate the probabilities.

APPENDIX XI. SOURCE LISTINGS OF THE CORE ALGORITHMS

The listings below reproduce the source code of the modules that implement the operations described in Materials and Methods, as of the commit of the repository from which the results were produced. The complete code, with the remaining modules and the tests, is in the repository.

Listing: src/leukemia_pp/stain.py

```
"""Reinhard colour normalisation in true (float) CIELAB.

Improvements over the textbook version used in v1:
* statistics come from *foreground* pixels (cells), not the whole frame, so the
  amount of
  empty background in a field no longer changes how strongly cell colour is
  rescaled;
* the per-channel gain is clipped, and frames with too little foreground are left
  unchanged (and flagged) instead of being normalised on unreliable statistics;
* the fitted reference is a first-class object that is saved/loaded, so
  validation, test
  and future inference images are normalised with the *training* reference.
"""
from __future__ import annotations

import json
from collections.abc import Iterable
from dataclasses import dataclass
from pathlib import Path

import cv2
import numpy as np
from skimage.filters import threshold_otsu

from .config import StainConfig


def bgr_to_lab(bgr: np.ndarray) -> np.ndarray:
    """uint8 BGR -> float32 Lab with L in [0,100], a/b signed (neutral = 0)."""
    return cv2.cvtColor(bgr.astype(np.float32) / 255.0, cv2.COLOR_BGR2Lab)


def lab_to_bgr(lab: np.ndarray) -> np.ndarray:
    bgr = cv2.cvtColor(lab.astype(np.float32), cv2.COLOR_Lab2BGR)
    return np.clip(np rint(bgr * 255.0), 0, 255).astype(np.uint8)


def otsu(values: np.ndarray) -> float:
    """Otsu threshold that tolerates constant input."""
    values = np.asarray(values).ravel()
    if values.size == 0 or float(values.max() - values.min()) < 1e-6:
        return float(values.mean()) if values.size else 0.0
    return float(threshold_otsu(values))


MIN_L_SPREAD = 1.0 # L* std below this = blank/flat frame, nothing to normalise
ANNOTATION_L = 8.0 # near-black pixels (scale bars, label text) are overlays, not
tissue
```

```

def foreground_stats(lab: np.ndarray, min_frac: float) -> tuple[np.ndarray,
np.ndarray] | None:
    """Per-channel (mean, std) over tissue pixels darker than the Otsu split of L*.

    Near-black pixels ( $L^* < \text{ANNOTATION\_L}$ ) are annotation overlays such as the black
    "200 pix"
    scale bar, and are excluded: otherwise the bar's size changes the statistics
    and thereby
    the colour of every cell in the image, and the bar style is class-specific in
    this dataset.

    Returns None when the frame is blank or the foreground covers less than
    `min_frac` of it.
    There is deliberately no whole-frame fallback: the reference is a *foreground*
    statistic,
    so matching a sparse field's mostly-background statistics to it would map pale
    background
    onto cell colour."""
    L = lab[..., 0]
    valid = L >= ANNOTATION_L
    if int(valid.sum()) < 0.5 * L.size or float(L[valid].std()) < MIN_L_SPREAD:
        return None
    fg = valid & (L < otsu(L[valid]))
    if fg.mean() < min_frac:
        return None
    px = lab[fg]
    return px.mean(axis=0), px.std(axis=0)

@dataclass(frozen=True)
class ReinhardReference:
    mean: tuple[float, float, float]
    std: tuple[float, float, float]
    n_images: int

    def to_dict(self) -> dict:
        return {"method": "reinhard-lab-foreground", "mean": list(self.mean),
                "std": list(self.std), "n_images": self.n_images}

    @classmethod
    def from_dict(cls, d: dict) -> ReinhardReference:
        return cls(tuple(d["mean"]), tuple(d["std"]), int(d["n_images"]))

    def save(self, path: Path) -> None:
        Path(path).write_text(json.dumps(self.to_dict(), indent=2), encoding="utf-
8")

    @classmethod
    def load(cls, path: Path) -> ReinhardReference:
        return cls.from_dict(json.loads(Path(path).read_text(encoding="utf-8")))

def fit_reference(images: Iterable[np.ndarray], cfg: StainConfig) ->
ReinhardReference:
    means, stds = [], []
    for bgr in images:
        stats = foreground_stats(bgr_to_lab(bgr), cfg.min_foreground_frac)
        if stats is None:
            # too sparse to be informative; would bias the
            reference
            continue
        means.append(stats[0])

```

```

        stds.append(stats[1])
    if not means:
        raise ValueError("no usable reference images (all blank or with too little
foreground)")
    return ReinhardReference(
        mean=tuple(float(x) for x in np.mean(means, axis=0)),
        std=tuple(float(x) for x in np.mean(stds, axis=0)),
        n_images=len(means),
    )

def normalize(bgr: np.ndarray, ref: ReinhardReference, cfg: StainConfig
) -> tuple[np.ndarray, bool]:
    """Returns (image, applied). `applied` is False, and the image is returned
unchanged,
when the frame has too little foreground to estimate its colour statistics."""
    lab = bgr_to_lab(bgr)
    stats = foreground_stats(lab, cfg.min_foreground_frac)
    if stats is None:
        return bgr.copy(), False
    src_mean, src_std = stats
    gain = np.clip(np.asarray(ref.std) / np.maximum(src_std, 1e-3),
*cfg.scale_clip)
    out = (lab - src_mean) * gain + np.asarray(ref.mean)
    out[..., 0] = np.clip(out[..., 0], 0, 100.0)
    return lab_to_bgr(out), True

```

Listing: src/leukemia_pp/segmentation.py

```

"""WBC instance segmentation (cells + nuclei).

Why colour, not darkness: in Romanowsky-stained smears, red blood cells and the
background
are pale and grey-blue, while leukocytes (and especially blasts) are strongly
magenta/purple.
That is a large separation on the Lab a* axis, whereas on grayscale intensity RBCs
overlap
with WBCs, which is why a global intensity Otsu merges all RBCs into the "cell"
mask.

Pipeline
1. a* score, lightly smoothed; threshold = max(Otsu, a_floor) -> foreground
2. fill holes, open, drop specks
3. split touching cells: distance transform + h-maxima markers + watershed. A
split is only
    accepted if every resulting piece is a plausible cell (>= min_cell_area)
4. per-cell nucleus: Otsu on L* over each cell's *interior* (rim excluded, since
smoothing
    makes the rim brighter), accepted only if the nucleus and cytoplasm classes
differ by
    >= nucleus_min-contrast L* units
"""
from __future__ import annotations

from dataclasses import dataclass, field

import cv2
import numpy as np
from scipy import ndimage as ndi
from skimage.measure import regionprops
from skimage.morphology import disk, h_maxima, opening
from skimage.segmentation import relabel_sequential, watershed

```

```

from .config import SegmentationConfig
from .stain import otsu

@dataclass(frozen=True)
class NucleusInfo:
    found: bool
    threshold: float          # L* split inside the cell
    contrast: float           # L* gap between cytoplasm-class mean and nucleus-class
    mean

@dataclass
class Segmentation:
    cell_labels: np.ndarray    # int32, 0 = background, 1..N = cell instances
    nucleus_labels: np.ndarray # int32, same ids as cell_labels
    a_score: np.ndarray        # float32 smoothed a*
    a_threshold: float
    nucleus_info: dict[int, NucleusInfo] = field(default_factory=dict)

    @property
    def cell_mask(self) -> np.ndarray:
        return self.cell_labels > 0

    @property
    def nucleus_mask(self) -> np.ndarray:
        return self.nucleus_labels > 0

    @property
    def n_cells(self) -> int:
        return int(self.cell_labels.max())

def _smooth(img: np.ndarray, sigma: float) -> np.ndarray:
    return cv2.GaussianBlur(img, (0, 0), sigma) if sigma > 0 else img

def _remove_small(mask: np.ndarray, min_area: int) -> np.ndarray:
    lab, n = ndi.label(mask)
    if n == 0:
        return mask
    areas = np.bincount(lab.ravel())
    keep = areas >= min_area
    keep[0] = False
    return keep[lab]

def split_touching(mask: np.ndarray, h: float, smooth: float, min_area: int) ->
np.ndarray:
    """Connected components -> instance labels, splitting necked-together cells."""
    cc, n = ndi.label(mask)
    out = np.zeros(mask.shape, np.int32)
    next_id = 1
    for idx, sl in enumerate(ndi.find_objects(cc), start=1):
        comp = np.pad(cc[sl] == idx, 1)
        dist = _smooth(ndi.distance_transform_edt(comp).astype(np.float32), smooth)
        markers, k = ndi.label(h_maxima(dist, h))
        pieces = comp.astype(np.int32)
        if k > 1:
            ws = watershed(-dist, markers, mask=comp)
            if np.bincount(ws.ravel())[1:].min() >= min_area:
                pieces = ws
        pieces = pieces[1:-1, 1:-1]

```

```

        for lab_id in np.unique(pieces[pieces > 0]):
            region = out[sl]
            region[pieces == lab_id] = next_id
            next_id += 1
    return out

def segment_cells(a_score: np.ndarray, cfg: SegmentationConfig) ->
tuple[np.ndarray, float]:
    thr = max(otsu(a_score), cfg.a_floor)
    mask = a_score > thr
    if cfg.open_radius > 0:
        mask = opening(mask, disk(cfg.open_radius))
    mask = _remove_small(mask, cfg.min_cell_area)
    mask = ndi.binary_fill_holes(mask)
    labels = split_touching(mask, cfg.split_h, cfg.split_smooth, cfg.min_cell_area)
    labels, _, _ = relabel_sequential(labels)
    return labels.astype(np.int32), float(thr)

def _class_split(values: np.ndarray) -> tuple[float, float]:
    """Otsu split of `values`; returns (threshold, mean gap between the two
    classes)."""
    thr = otsu(values)
    lo, hi = values[values < thr], values[values >= thr]
    if lo.size == 0 or hi.size == 0:
        return thr, 0.0
    return thr, float(hi.mean() - lo.mean())

def segment_nuclei(L: np.ndarray, cell_labels: np.ndarray, cfg: SegmentationConfig
) -> tuple[np.ndarray, dict[int, NucleusInfo]]:
    Ls = _smooth(L.astype(np.float32), cfg.nucleus_smooth_sigma)
    nucleus = np.zeros(cell_labels.shape, np.int32)
    info: dict[int, NucleusInfo] = {}
    for region in regionprops(cell_labels):
        sl, cid = region.slice, region.label
        cell = cell_labels[sl] == cid
        interior = ndi.binary_erosion(cell, disk(cfg.nucleus_rim_erode)) if
cfg.nucleus_rim_erode \
        else cell
        if interior.sum() < max(cfg.nucleus_min_area, 1):
            info[cid] = NucleusInfo(False, float("nan"), 0.0)
            continue
        thr, contrast = _class_split(Ls[sl][interior]) # statistics from the
interior only
        if contrast < cfg.nucleus_min_contrast:
            info[cid] = NucleusInfo(False, thr, contrast)
            continue
        nuc = cell & (Ls[sl] < thr)
        nuc = ndi.binary_fill_holes(opening(nuc, disk(1)))
        lab, n = ndi.label(nuc)
        if n:
            areas = np.bincount(lab.ravel())
            areas[0] = 0
            keep = areas >= max(cfg.nucleus_min_area, cfg.nucleus_keep_frac *
areas.max())
            keep[0] = False
            nuc = keep[lab] & cell
            ok = nuc.sum() >= max(cfg.nucleus_min_area, cfg.nucleus_min_cell_frac *
cell.sum())
            info[cid] = NucleusInfo(bool(ok), thr, contrast)
            if ok:

```

```

        nucleus[sl][nuc] = cid
    return nucleus, info

```

```

def segment(lab: np.ndarray, cfg: SegmentationConfig) -> Segmentation:
    """`lab` is a float Lab image (L 0-100, a/b signed), e.g. from
    `stain.bgr_to_lab`. """
    a_score = _smooth(lab[..., 1].astype(np.float32), cfg.smooth_sigma)
    cell_labels, thr = segment_cells(a_score, cfg)
    nucleus_labels, info = segment_nuclei(lab[..., 0], cell_labels, cfg)
    return Segmentation(cell_labels, nucleus_labels, a_score, thr, info)

```

Listing: src/leukemia_pp/hds.py

```

"""Stabilised Hybrid Diffusion-Steered (HDS) denoising + edge indicator.

```

The Block-2 prototype (and v1 `ml\_pipeline/core.py`) uses the conduction coefficient

```

    c(s) = g * ( 1/(1 + s^2/h^2)  +  1/(|s| + eps) )          (Perona-Malik + total
variation)

```

The TV term reaches $1/\epsilon = 1e8$ in flat regions, so the explicit update $u += dt * (\text{div}(c \text{ grad } u) - \lambda * \dots)$ violates the stability bound $dt \leq 1/(4 * c_{\max})$ by orders of magnitude and diverges on non-speckle noise (documented in docs/paper.pdf).

This version keeps the *hybrid* idea but is unconditionally stable:

```

* both diffusivities are normalised to (0, 1]:
    PM(s) = 1/(1 + (s/h)^2)          TV_beta(s) = beta / sqrt(s^2 + beta^2)
(Charbonnier)
    c(s) = w*PM(s) + (1 - w)*TV_beta(s)    so c_max = 1 and dt <= 0.25 is
stable;
* the data term is the additive-noise fidelity -lambda*(u - f) (the legacy log-
prior is a
multiplicative-speckle model and is the second source of divergence on this
data);
* iteration stops early once the mean update falls below `tol`.
Because every update is a convex combination of neighbouring values plus a pull
towards f,
the discrete maximum principle holds: output stays within [min(f), max(f)].

```

The edge indicator is $1 - \text{mean}_k c(|\text{grad}_k u|)$ of the *denoised* image: ~ 0 in flat regions, approaching 1 across strong edges, on a fixed (not per-image) scale.

```

from __future__ import annotations

```

```

import numpy as np

```

```

from .config import HDSConfig

```

```

def _diffusivity(s: np.ndarray, h: float, beta: float, w: float) -> np.ndarray:
    return w / (1.0 + (s / h) ** 2) + (1.0 - w) * beta / np.sqrt(s * s + beta *
beta)

```

```

def _neighbour_diffs(u: np.ndarray):
    p = np.pad(u, 1, mode="edge")
    return (p[:-2, 1:-1] - u, p[2:, 1:-1] - u, p[1:-1, :-2] - u, p[1:-1, 2:] - u)
# N S W E

```

```

def hds_denoise(f: np.ndarray, cfg: HDSConfig) -> tuple[np.ndarray, np.ndarray,
int]:
    """Denoise a single-channel float image `f` (intensity roughly in [0, 1]).

    Returns (u, edge, n_iterations)."""
    f = np.asarray(f, dtype=np.float64)
    u = f.copy()
    n_done = 0
    for n_done in range(1, cfg.iterations + 1):
        diffs = _neighbour_diffs(u)
        flux = sum(_diffusivity(np.abs(d), cfg.h, cfg.beta, cfg.hybrid_weight) * d
for d in diffs)
        u_next = u + cfg.dt * (flux - cfg.lam * (u - f))
        converged = float(np.abs(u_next - u).mean()) < cfg.tol
        u = u_next
        if converged:
            break
        diffs = _neighbour_diffs(u)
        c_mean = sum(_diffusivity(np.abs(d), cfg.h, cfg.beta, cfg.hybrid_weight) for d
in diffs) / 4.0
        edge = np.clip(1.0 - c_mean, 0.0, 1.0)
        return u.astype(np.float32), edge.astype(np.float32), n_done

```

Listing: src/leukemia_pp/neutralise.py

```

"""Shortcut removal: a 'clean' view of the image that contains only the WBCs.

Everything outside the dilated cell mask (background, RBCs, debris, scale bar /
text labels)
is replaced by one constant neutral colour, so a network cannot classify on slide
background
tint, scale-bar style or RBC appearance. With `white_balance`, the colour cast of
the slide is
also removed from the *cells*: a*/b* are shifted and L* scaled so the measured
background
becomes the neutral fill colour.

Two outputs, so colour can be ablated: `rgb_clean` (uint8 RGB) and `gray_clean`
(L*/100, float32). Cell colour can still carry stain/session information even after
white
balance: quantify it with `leukemia-pp audit`, and consider training on
`gray_clean` and/or
with colour augmentation.
"""
from __future__ import annotations

import numpy as np
from scipy import ndimage as ndi

from .config import NeutraliseConfig
from .segmentation import Segmentation
from .stain import bgr_to_lab, lab_to_bgr

def keep_mask(seg: Segmentation, cfg: NeutraliseConfig) -> np.ndarray:
    mask = seg.cell_mask
    if cfg.dilate_px > 0 and mask.any():
        mask = ndi.binary_dilation(mask, iterations=cfg.dilate_px)
    return mask

```

```

def background_colour(lab: np.ndarray, keep: np.ndarray, min_px: int) -> np.ndarray
| None:
    """Median Lab of the brighter half of the non-cell pixels (slide background,
not RBC rims
or debris); None if there is too little background to estimate it."""
    bg = ~keep
    if int(bg.sum()) < min_px:
        return None
    L = lab[..., 0]
    candidate = bg & (L >= np.percentile(L[bg], 50))
    return np.median(lab[candidate], axis=0)

def neutralise(norm_bgr: np.ndarray, seg: Segmentation, cfg: NeutraliseConfig
) -> tuple[np.ndarray, np.ndarray]:
    """Returns (rgb_clean uint8 HxWx3, gray_clean float32 HxW)."""
    lab = bgr_to_lab(norm_bgr)
    keep = keep_mask(seg, cfg)
    if cfg.white_balance:
        bg = background_colour(lab, keep, cfg.min_background_px)
        if bg is not None:
            gain = float(np.clip(cfg.fill_L / max(float(bg[0]), 1e-3),
*cfg.gain_clip))
            lab[..., 0] = np.clip(lab[..., 0] * gain, 0.0, 100.0)
            lab[..., 1] -= bg[1]
            lab[..., 2] -= bg[2]
    lab[~keep] = (cfg.fill_L, 0.0, 0.0)
    rgb = np.ascontiguousarray(lab_to_bgr(lab)[..., :-1])
    return rgb, (lab[..., 0] / 100.0).astype(np.float32)

```

Listing: src/leukemia_ml/models/heads.py

```

"""Pooling / classification heads over a bag of cell embeddings.

Bags are padded to (B, K, D) with a boolean mask (B, K), True = real cell.

* `GatedAttentionMIL` (Ilse, Tomczak, Welling 2018):  $a_k = \text{softmax}_k(w^T [\tanh(V h_k) * \text{sigm}(U h_k)])$ ,
bag =  $\sum_k a_k h_k$ . The softmax normalises over the *valid* cells only, so the
bag embedding
does not depend on how many cells there are (cell count is a class-correlated
density
shortcut in this data).
* `MeanPoolHead`: mean of valid cells, the permutation-invariant baseline.
* `CellHead`: classify each cell independently (cell-level training / evaluation).
"""
from __future__ import annotations

import torch
import torch.nn as nn

def masked_softmax(logits: torch.Tensor, mask: torch.Tensor) -> torch.Tensor:
    logits = logits.masked_fill(~mask, float("-inf"))
    out = torch.softmax(logits, dim=1)
    return torch.nan_to_num(out, nan=0.0) # an all-masked bag gives zeros,
not NaN

class GatedAttentionMIL(nn.Module):
    def __init__(self, in_dim: int, n_classes: int, hidden: int = 256, attn_dim:
int = 128,
                dropout: float = 0.2):

```

```

        super().__init__()
        self.proj = nn.Sequential(nn.Linear(in_dim, hidden), nn.GELU(),
nn.Dropout(dropout))
        self.V = nn.Linear(hidden, attn_dim)
        self.U = nn.Linear(hidden, attn_dim)
        self.w = nn.Linear(attn_dim, 1)
        self.cls = nn.Linear(hidden, n_classes)

    def forward(self, h: torch.Tensor, mask: torch.Tensor):
        z = self.proj(h)
        a = self.w(torch.tanh(self.V(z)) * torch.sigmoid(self.U(z))).squeeze(-1)
        attn = masked_softmax(a, mask)
        bag = (attn.unsqueeze(-1) * z).sum(dim=1)
        return self.cls(bag), attn

class MeanPoolHead(nn.Module):
    def __init__(self, in_dim: int, n_classes: int, hidden: int = 256, dropout:
float = 0.2):
        super().__init__()
        self.proj = nn.Sequential(nn.Linear(in_dim, hidden), nn.GELU(),
nn.Dropout(dropout))
        self.cls = nn.Linear(hidden, n_classes)

    def forward(self, h: torch.Tensor, mask: torch.Tensor):
        z = self.proj(h)
        w = mask.float()
        denom = w.sum(dim=1, keepdim=True).clamp_min(1.0)
        attn = w / denom
        return self.cls((attn.unsqueeze(-1) * z).sum(dim=1)), attn

class CellHead(nn.Module):
    """Per-cell logits; the bag logit is the mean of the cell logits (so one code
path)."""
    def __init__(self, in_dim: int, n_classes: int, hidden: int = 256, dropout:
float = 0.2):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(in_dim, hidden), nn.GELU(),
nn.Dropout(dropout),
                                nn.Linear(hidden, n_classes))

    def forward(self, h: torch.Tensor, mask: torch.Tensor):
        logits = self.net(h)
        w = mask.float()
        attn = w / w.sum(dim=1, keepdim=True).clamp_min(1.0)
        return (attn.unsqueeze(-1) * logits).sum(dim=1), attn

def build_head(kind: str, in_dim: int, n_classes: int, hidden: int, dropout: float)
-> nn.Module:
    if kind == "mil":
        return GatedAttentionMIL(in_dim, n_classes, hidden, dropout=dropout)
    if kind == "mean":
        return MeanPoolHead(in_dim, n_classes, hidden, dropout)
    if kind == "cell":
        return CellHead(in_dim, n_classes, hidden, dropout)
    raise ValueError(f"unknown head {kind!r}")

```

Listing: src/leukemia_ml/eval/folds.py

```

"""Repeated, class-stratified, GROUP-disjoint cross-validation folds."""

```

```

from __future__ import annotations

import numpy as np

def grouped_folds(groups: np.ndarray, y: np.ndarray, n_splits: int = 5, seeds=(0,
1, 2)):
    """Yield (seed, fold, train_idx, test_idx). No group ever appears in both train
and test;
    classes are balanced across folds as well as the group structure allows."""
    from sklearn.model_selection import StratifiedGroupKFold
    for seed in seeds:
        cv = StratifiedGroupKFold(n_splits=n_splits, shuffle=True,
random_state=seed)
        for fold, (tr, te) in enumerate(cv.split(np.zeros(len(y)), y, groups)):
            assert not set(groups[tr]) & set(groups[te])
            yield seed, fold, tr, te

def purge_train(train_idx: np.ndarray, test_idx: np.ndarray, y: np.ndarray, order:
np.ndarray,
                embargo: int) -> np.ndarray:
    """Remove from `train_idx` every sample whose sequence number is within
`embargo` of a test
    sample of the SAME class. Acquisition sessions are contiguous in the numbering,
so this
    removes training samples that share a session/patient with a test sample even
when the
    group blocks do not line up with the true patient boundaries."""
    if embargo <= 0 or len(test_idx) == 0:
        return train_idx
    keep = np.ones(len(train_idx), bool)
    for c in np.unique(y[test_idx]):
        t = np.sort(order[test_idx[y[test_idx] == c]])
        tr_mask = y[train_idx] == c
        pos = np.searchsorted(t, order[train_idx[tr_mask]])
        lo = np.abs(order[train_idx[tr_mask]] - t[np.clip(pos - 1, 0, len(t) - 1)])
        hi = np.abs(order[train_idx[tr_mask]] - t[np.clip(pos, 0, len(t) - 1)])
        keep[np.nonzero(tr_mask)[0][np.minimum(lo, hi) <= embargo]] = False
    return train_idx[keep]

def contiguous_folds(order: np.ndarray, y: np.ndarray, n_splits: int = 5, embargo:
int = 0,
                    rotate: int = 0):
    """Session-aware folds: within each class, sort by sequence number and cut the
sequence into
    `n_splits` CONSECUTIVE segments; fold k tests segment k of every class.

    Images are numbered in capture order and sessions/patients are contiguous in
that numbering,
    so a test segment shares sessions with training images only at its two edges
(at most one
    patient per edge); `embargo` then removes training images within that many
numbers of a test
    image. Contrast with random assignment of fixed blocks, where almost every
block has a
    same-patient neighbour in the training set. `rotate` shifts segment boundaries
(a different
    but equally valid partition) so repeats are possible."""
    fold_of = np.zeros(len(y), int)
    for c in np.unique(y):
        members = np.nonzero(y == c)[0]

```

```

        ranked = members[np.argsort(order[members], kind="stable")]
        shift = (rotate * len(ranked)) // (n_splits * 3 if n_splits else 1)
        ranked = np.roll(ranked, -shift)
        for k, seg in enumerate(np.array_split(np.arange(len(ranked)), n_splits)):
            fold_of[ranked[seg]] = k
    for k in range(n_splits):
        te = np.nonzero(fold_of == k)[0]
        tr = np.nonzero(fold_of != k)[0]
        yield rotate, k, purge_train(tr, te, y, order, embargo), te

```

Listing: src/leukemia_ml/eval/stats.py

```

"""Uncertainty and significance for classifier comparison.

Why not plain image-level statistics: images of one patient/session are strongly
correlated,
so resampling images treats ~3,000 correlated items as independent and gives
intervals that
are far too narrow. We resample *groups* (cluster bootstrap). For repeated k-fold
CV the usual
paired t-test is miscalibrated because training sets overlap; the Nadeau-Bengio
correction
inflates the variance accordingly.
"""
from __future__ import annotations

import math
from collections.abc import Callable

import numpy as np

def cluster_bootstrap(metric: Callable[[np.ndarray], float], groups: np.ndarray,
n_boot: int = 2000,
                    seed: int = 0, alpha: float = 0.05) -> dict[str, float]:
    """`metric(idx)` evaluates the metric on the sample indices `idx`; groups are
    resampled
    with replacement and every member of a drawn group enters the sample."""
    rng = np.random.default_rng(seed)
    uniq, inverse = np.unique(groups, return_inverse=True)
    members = [np.nonzero(inverse == g)[0] for g in range(len(uniq))]
    vals = np.empty(n_boot)
    for b in range(n_boot):
        drawn = rng.integers(0, len(uniq), len(uniq))
        vals[b] = metric(np.concatenate([members[g] for g in drawn]))
    vals = vals[np.isfinite(vals)]
    point = float(metric(np.arange(len(groups))))
    lo, hi = np.quantile(vals, [alpha / 2, 1 - alpha / 2])
    return {"point": point, "lo": float(lo), "hi": float(hi), "se":
float(vals.std(ddof=1)),
        "n_groups": int(len(uniq))}

def paired_bootstrap(metric_a: Callable[[np.ndarray], float], metric_b:
Callable[[np.ndarray], float],
                    groups: np.ndarray, n_boot: int = 2000, seed: int = 0) ->
dict[str, float]:
    """Difference metric_a - metric_b on identical resampled groups; two-sided p
    from the
    bootstrap distribution of the difference (share of draws on the other side of
    zero)."""
    rng = np.random.default_rng(seed)
    uniq, inverse = np.unique(groups, return_inverse=True)

```

```

members = [np.nonzero(inverse == g)[0] for g in range(len(uniq))]
diffs = np.empty(n_boot)
for b in range(n_boot):
    idx = np.concatenate([members[g] for g in rng.integers(0, len(uniq),
len(uniq))])
    diffs[b] = metric_a(idx) - metric_b(idx)
    diffs = diffs[np.isfinite(diffs)]
    point = float(metric_a(np.arange(len(groups))) -
metric_b(np.arange(len(groups))))
    p = 2 * min((diffs <= 0).mean(), (diffs >= 0).mean())
    lo, hi = np.quantile(diffs, [0.025, 0.975])
    return {"diff": point, "lo": float(lo), "hi": float(hi), "p": float(min(1.0,
max(p, 1 / n_boot)))}

def corrected_resampled_ttest(diffs: np.ndarray, n_train: int, n_test: int) ->
dict[str, float]:
    """Nadeau & Bengio (2003):  $t = \text{mean}(d) / \sqrt{(1/J + n_{\text{test}}/n_{\text{train}}) * \text{var}(d)}$ ."""
    from scipy import stats
    d = np.asarray(diffs, float)
    j = len(d)
    var = d.var(ddof=1)
    if j < 2 or var == 0:
        return {"t": float("nan"), "p": float("nan"), "mean": float(d.mean()) if j
else float("nan")}
    t = d.mean() / math.sqrt((1.0 / j + n_test / n_train) * var)
    return {"t": float(t), "p": float(2 * stats.t.sf(abs(t), j - 1)), "mean":
float(d.mean())}

def holm(pvals: list[float]) -> list[float]:
    """Holm-Bonferroni adjusted p-values (same order as input)."""
    m = len(pvals)
    order = np.argsort(pvals)
    adj = np.empty(m)
    running = 0.0
    for rank, i in enumerate(order):
        running = max(running, (m - rank) * pvals[i])
        adj[i] = min(1.0, running)
    return adj.tolist()

```

Listing: src/leukemia_ml/eval/calibration.py

```

"""Temperature scaling (Guo et al. 2017): fit one scalar T on VALIDATION logits."""
from __future__ import annotations

import numpy as np
import torch
import torch.nn.functional as F

def fit_temperature(logits: np.ndarray, y: np.ndarray, iters: int = 200) -> float:
    z = torch.tensor(logits, dtype=torch.float64)
    t = torch.tensor(y, dtype=torch.long)
    log_t = torch.zeros(1, dtype=torch.float64, requires_grad=True)
    opt = torch.optim.LBFGS([log_t], lr=0.1, max_iter=iters)

    def closure():
        opt.zero_grad()
        loss = F.cross_entropy(z / log_t.exp(), t)
        loss.backward()
        return loss

```

```

opt.step(closure)
return float(log_t.exp().item())

```

```

def apply_temperature(logits: np.ndarray, temperature: float) -> np.ndarray:
    z = logits / temperature
    z = z - z.max(axis=1, keepdims=True)
    e = np.exp(z)
    return e / e.sum(axis=1, keepdims=True)

```

Listing: src/leukemia_ml/eval/metrics.py

```

"""Classification metrics for image-level predictions (probabilities (N,C), labels
(N,))."""
from __future__ import annotations

import numpy as np

def confusion(y: np.ndarray, pred: np.ndarray, n_classes: int) -> np.ndarray:
    m = np.zeros((n_classes, n_classes), np.int64)
    np.add.at(m, (y, pred), 1)
    return m

def per_class_recall(cm: np.ndarray) -> np.ndarray:
    support = cm.sum(axis=1)
    return np.divide(np.diag(cm), support, out=np.full(len(cm), np.nan),
where=support > 0)

def balanced_accuracy(y: np.ndarray, pred: np.ndarray, n_classes: int) -> float:
    r = per_class_recall(confusion(y, pred, n_classes))
    return float(np.nanmean(r))

def macro_f1(y: np.ndarray, pred: np.ndarray, n_classes: int) -> float:
    cm = confusion(y, pred, n_classes)
    tp = np.diag(cm).astype(float)
    prec = np.divide(tp, cm.sum(axis=0), out=np.zeros(n_classes),
where=cm.sum(axis=0) > 0)
    rec = np.divide(tp, cm.sum(axis=1), out=np.zeros(n_classes),
where=cm.sum(axis=1) > 0)
    f1 = np.divide(2 * prec * rec, prec + rec, out=np.zeros(n_classes), where=(prec
+ rec) > 0)
    present = cm.sum(axis=1) > 0
    return float(f1[present].mean())

def auroc_ovr(y: np.ndarray, probs: np.ndarray) -> float:
    """Macro one-vs-rest AUROC (rank-based, ties averaged); classes absent in `y`
are skipped."""
    from scipy.stats import rankdata
    aucs = []
    for c in range(probs.shape[1]):
        pos = y == c
        n_pos, n_neg = int(pos.sum()), int((~pos).sum())
        if n_pos == 0 or n_neg == 0:
            continue
        ranks = rankdata(probs[:, c])
        aucs.append((ranks[pos].sum() - n_pos * (n_pos + 1) / 2) / (n_pos * n_neg))
    return float(np.mean(aucs)) if aucs else float("nan")

```

```

def ece(y: np.ndarray, probs: np.ndarray, n_bins: int = 15) -> float:
    """Expected calibration error of the top-label confidence."""
    conf, pred = probs.max(axis=1), probs.argmax(axis=1)
    edges = np.linspace(0, 1, n_bins + 1)
    total = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            total += m.mean() * abs((pred[m] == y[m]).mean() - conf[m].mean())
    return float(total)

def binary_sens_spec(y: np.ndarray, pred: np.ndarray, benign_class: int = 0) -> dict[str, float]:
    """Benign-vs-malignant sensitivity/specificity from multi-class labels
    (malignant = not benign)."""
    pos, pred_pos = y != benign_class, pred != benign_class
    sens = float((pred_pos & pos).sum() / max(pos.sum(), 1))
    spec = float((~pred_pos & ~pos).sum() / max((~pos).sum(), 1))
    return {"sensitivity": sens, "specificity": spec}

def summarize(y: np.ndarray, probs: np.ndarray, benign_class: int = 0) -> dict:
    n_classes = probs.shape[1]
    pred = probs.argmax(axis=1)
    cm = confusion(y, pred, n_classes)
    return {
        "n": int(len(y)),
        "accuracy": float((pred == y).mean()),
        "balanced_accuracy": balanced_accuracy(y, pred, n_classes),
        "macro_f1": macro_f1(y, pred, n_classes),
        "auroc_ovr": auroc_ovr(y, probs),
        "ece": ece(y, probs),
        "per_class_recall": [None if np.isnan(r) else float(r) for r in
per_class_recall(cm)],
        "confusion": cm.tolist(),
        **binary_sens_spec(y, pred, benign_class),
    }

```

APPENDIX XII. TEST SUITE

The repository has 189 automated tests, all of which passed in the run used for this paper (python -m pytest, 354 seconds on a CPU). They are listed by file; each name states the behaviour that is checked.

```
test_audit.py::test_audit_flags_background_and_scale_bar_shortcuts
test_audit.py::test_clean_image_carries_no_background_or_corner_signal
test_audit.py::test_white_balance_removes_cell_colour_cast
test_audit.py::test_audit_writes_reports_and_cli
test_audit.py::test_verdict_thresholds
test_audit.py::test_corner_patch_shape_and_range
test_config.py::test_roundtrip_and_hash_stable
test_config.py::test_partial_override_and_hash_changes
test_config.py::test_invalid_config_rejected[bad0]
test_config.py::test_invalid_config_rejected[bad1]
test_config.py::test_invalid_config_rejected[bad2]
test_config.py::test_invalid_config_rejected[bad3]
test_emd.py::test_reconstruction_is_exact
test_emd.py::test_modes_are_ordered_fine_to_coarse
test_emd.py::test_characteristic_scale_recovers_period
test_emd.py::test_deterministic_and_seed_sensitive
test_emd.py::test_pure_selection_rejects_noise_and_drift_bands
test_emd.py::test_pure_is_sum_of_selected_modes
test_emd.py::test_flat_and_tiny_inputs_do_not_crash
test_emd.py::test_ensemble_noise_cancels_in_the_mean
test_emd.py::test_config_validation
test_emd.py::test_white_noise_mode_is_rejected_but_cell_scale_structure_is_kept
test_features.py::test_disc_geometry
test_features.py::test_elongated_cell_has_lower_circularity_and_higher_aspect
test_features.py::test_border_cells_flagged
test_features.py::test_missing_nucleus_gives_nan_not_zero
test_features.py::test_glcml_ignores_surround
test_features.py::test_glcml_separates_smooth_from_textured
test_features.py::test_glcml_tiny_region_is_nan
test_features.py::test_aggregate_prefers_interior_cells
test_features.py::test_aggregate_empty_image
test_hds.py::test_denoises_additive_noise[0.05]
test_hds.py::test_denoises_additive_noise[0.08]
test_hds.py::test_denoises_additive_noise[0.12]
test_hds.py::test_stable_and_bounded_even_when_run_to_exhaustion
test_hds.py::test_legacy_hds_collapses_on_the_same_data
test_hds.py::test_constant_image_is_a_fixed_point_with_no_edges
test_hds.py::test_edge_indicator_fixed_scale_and_localised
test_hds.py::test_early_stopping
test_hds.py::test_invalid_params_rejected[bad0]
test_hds.py::test_invalid_params_rejected[bad1]
test_hds.py::test_invalid_params_rejected[bad2]
test_hds.py::test_invalid_params_rejected[bad3]
test_metrics.py::test_dice_iou
test_ml_data.py::test_index_matches_cells_csv_and_never_splits_a_group
test_ml_data.py::test_index_errors
test_ml_data.py::test_cut_crop_geometry_isolation_and_padding
test_ml_data.py::test_crop_cache_build_reuse_and_key
test_ml_data.py::test_crop_sources_differ_as_designed
test_ml_data.py::test_isolated_crops_leak_neither_scale_bar_nor_background_class
test_ml_data.py::test_augment_is_deterministic_shape_preserving_and_keeps_fill
test_ml_data.py::test_bag_dataset_caps_cells_is_deterministic_and_collates
```

```

test_ml_data.py::test_feature_scaler_uses_train_statistics_and_imputes
test_ml_data.py::test_embedding_dataset_matches_index
test_ml_eval.py::test_metrics_known_values
test_ml_eval.py::test_auroc_and_ece
test_ml_eval.py::test_binary_sens_spec_and_summary
test_ml_eval.py::test_grouped_folds_are_group_disjoint_stratified_and_seeded
test_ml_eval.py::test_cluster_bootstrap_widens_with_correlated_groups
test_ml_eval.py::test_paired_bootstrap_detects_real_difference_and_not_null
test_ml_eval.py::test_corrected_ttest_is_more_conservative_than_plain
test_ml_eval.py::test_holm_matches_hand_computation
test_ml_eval.py::test_temperature_scaling_fixes_overconfidence
test_ml_eval.py::test_probe_finds_signal_and_not_noise
test_ml_eval.py::test_experiment_config_roundtrip_and_validation
test_ml_eval.py::test_purge_train_removes_same_class_neighbours_only
test_ml_eval.py::test_embargo_never_leaves_train_test_overlap_and_shrinks_train
test_ml_eval.py::test_contiguous_folds_partition_each_class_into_consecutive_segments
test_ml_eval.py::test_contiguous_embargo_removes_only_edge_neighbours
test_ml_eval.py::test_contiguous_rotation_changes_boundaries
test_ml_eval.py::test_probe_rejects_unknown_scheme
test_ml_experiments.py::test_apply_overrides_do_not_mutate_base
test_ml_experiments.py::test_probe_ablation_structure_ci_and_determinism
test_ml_experiments.py::test_markdown_mentions_reference_and_ci
test_ml_experiments.py::test_cli_ablate
test_ml_experiments.py::test_contiguous_is_the_default_scheme_and_is_recorded
test_ml_fullrun.py::test_contiguous_split_segments_embargo_and_coverage
test_ml_fullrun.py::test_split_config_validation
test_ml_fullrun.py::test_trainer_uses_contiguous_split_and_never_trains_on_heldout
test_ml_fullrun.py::test_interrupted_and_resumed_frozen_run_equals_uninterrupted
test_ml_fullrun.py::test_interrupted_and_resumed_lora_run_with_augmentation_is_exact
test_ml_fullrun.py::test_finished_run_is_not_recomputed_and_other_config_does_not_resume
test_ml_fullrun.py::test_amp_dtype_is_none_on_cpu
test_ml_fullrun.py::test_cv_tests_every_image_once_pools_and_resumes
test_ml_fullrun.py::test_cv_subset_of_folds
test_ml_fullrun.py::test_preflight_reports_timing_and_projects_epoch_time
test_ml_fullrun.py::test_bundle_roundtrip_and_corruption_detection
test_ml_models.py::test_lora_is_identity_at_init_and_merge_roundtrips
test_ml_models.py::test_lora_trains_adapters_only
test_ml_models.py::test_apply_lora_wraps_vit_attention_and_counts_params
test_ml_models.py::test_dinobloom_checkpoint_loads_exactly
test_ml_models.py::test_dinobloom_resamples_position_embedding
test_ml_models.py::test_dinobloom_errors
test_ml_models.py::test_cnn_encoder_and_trainable_modes
test_ml_models.py::test_masked_softmax_normalises_valid_cells_only
test_ml_models.py::test_mil_is_permutation_invariant_and_padding_independent
test_ml_models.py::test_mil_ignores_cell_count_duplication
test_ml_models.py::test_heads_backprop_everywhere_and_no_nan_on_empty_bag[GatedAttentionMIL]
test_ml_models.py::test_heads_backprop_everywhere_and_no_nan_on_empty_bag[MeanPoolHead]
test_ml_models.py::test_feature_branch_group_dropout_and_ablation
test_ml_models.py::test_model_forward_backward_and_param_groups
test_ml_models.py::test_frozen_encoder_builds_no_graph_and_lora_model_has_adapters
test_ml_models.py::test_pad_bags_layout
test_ml_models.py::test_model_config_validation
test_ml_train.py::test_class_weights_scheduler_subset
test_ml_train.py::test_stratified_subset_balances_classes
test_ml_train.py::test_embedding_cache_is_keyed_and_reused
test_ml_train.py::test_frozen_embedding_training_is_deterministic_and_learns
test_ml_train.py::test_different_seeds_change_the_run
test_ml_train.py::test_early_stopping_and_best_checkpoint

```

```

test_ml_train.py::test_lora_training_updates_adapters_only
test_ml_train.py::test_full_finetune_path_with_augmentation_runs
test_ml_train.py::test_binary_task_and_cell_head
test_ml_xai.py::test_grad_cam_shape_range_determinism[cnn_model-64]
test_ml_xai.py::test_grad_cam_shape_range_determinism[vit_model-56]
test_ml_xai.py::test_grad_cam_depends_on_the_input
test_ml_xai.py::test_randomisation_sanity_reports_each_stage_and_decorrelates
test_ml_xai.py::test_deletion_curve_structure_and_start_value
test_neutralise.py::test_everything_outside_cells_is_one_constant
test_neutralise.py::test_rbcs_and_background_colour_leave_no_trace
test_neutralise.py::test_scale_bar_label_is_removed
test_neutralise.py::test_cell_pixels_untouched_without_white_balance
test_neutralise.py::test_white_balance_cancels_slide_tint_in_cells
test_neutralise.py::test_no_cells_gives_uniform_image
test_neutralise.py::test_all_cell_frame_skips_white_balance_without_crashing
test_neutralise.py::test_pipeline_emits_clean_arrays_and_can_disable
test_neutralise.py::test_config_rejects_unknown[bad0]
test_pipeline.py::test_end_to_end_outputs
test_pipeline.py::test_stain_reference_fitted_on_train_only
test_pipeline.py::test_deterministic_across_runs_and_worker_counts
test_pipeline.py::test_corrupt_image_is_reported_not_fatal
test_pipeline.py::test_changed_config_in_same_output_dir_is_refused
test_pipeline.py::test_duplicate_image_ids_rejected
test_pipeline.py::test_unknown_class_folders_ignored_and_empty_input_errors
test_pipeline.py::test_sparse_field_is_flagged
test_pipeline.py::test_empty_field_is_flagged_no_cells
test_pipeline.py::test_stain_disabled_needs_no_reference
test_pipeline.py::test_max_side_downscales
test_pipeline.py::test_cli
test_pipeline.py::test_pipeline_module_exports_documented_columns
test_representations.py::test_default_config_computes_no_optional_arrays
test_representations.py::test_all_on_arrays_shapes_and_columns
test_representations.py::test_boundary_edge_stat_exceeds_interior_for_flat_cells
test_representations.py::test_pipeline_run_with_everything_enabled
test_representations.py::test_load_stack_scaling_and_order
test_representations.py::test_missing_channel_gives_actionable_error
test_representations.py::test_crop_cell_is_centred_square_and_deterministic
test_representations.py::test_crop_of_border_cell_is_padded_not_shifted
test_representations.py::test_presets_and_feature_groups_are_consistent
test_responses.py::test_uniform_image_has_no_response_anywhere_including_border
test_responses.py::test_blob_responds_and_range_is_fixed
test_responses.py::test_response_does_not_rescale_with_image_content
test_segmentation.py::test_rbcs_are_not_cells
test_segmentation.py::test_wbcs_found_among_rbcs
test_segmentation.py::test_touching_cells_are_split
test_segmentation.py::test_single_elongated_cell_is_not_oversplit
test_segmentation.py::test_empty_field_gives_nothing
test_segmentation.py::test_specks_are_dropped
test_segmentation.py::test_nucleus_found_inside_cell_with_dark_core
test_segmentation.py::test_flat_cell_reports_no_nucleus_instead_of_inventing_one
test_segmentation.py::test_labels_are_consecutive_and_nucleus_ids_match
test_segmentation.py::test_split_rejects_fragments_below_min_area
test_segmentation.py::test_noise_robustness[0]
test_segmentation.py::test_noise_robustness[1]
test_segmentation.py::test_noise_robustness[2]
test_splits.py::test_allocate[0-expected0]
test_splits.py::test_allocate[1-expected1]
test_splits.py::test_allocate[2-expected2]
test_splits.py::test_allocate[3-expected3]
test_splits.py::test_allocate[10-expected4]
test_splits.py::test_allocate[100-expected5]
test_splits.py::test_allocate_sums_for_all_sizes
test_splits.py::test_deterministic_and_stratified

```

```
test_splits.py::test_exact_duplicates_never_straddle_splits
test_splits.py::test_group_regex_keeps_patient_together
test_splits.py::test_every_class_has_train
test_splits.py::test_sequence_blocks_keep_consecutive_numbers_together
test_splits.py::test_duplicate_pair_straddling_a_block_boundary_stays_together
test_splits.py::test_blocks_are_per_class
test_splits.py::test_invalid_block_rejected
test_stain.py::test_normalisation_removes_global_tint
test_stain.py::test_self_reference_is_near_identity
test_stain.py::test_constant_image_does_not_explode
test_stain.py::test_reference_roundtrip
test_stain.py::test_foreground_not_dominated_by_background
test_stain.py::test_sparse_field_is_left_unchanged_not_misnormalised
test_stain.py::test_no_discontinuity_around_threshold
test_stain.py::test_scale_bar_does_not_change_normalisation
test_stain.py::test_annotation_pixels_excluded_from_reference_fit
```

APPENDIX XIII. FILES RELEASED WITH THE PAPER

Table A16. Result files and the scripts that produce them.

File	Content
docs/results/cv_dinobloom_s_frozen_mil.json	Headline cross-validation: pooled and per-fold metrics, intervals, temperatures (Table 4).
docs/results/cv_run/	Per-fold predictions, training logs and metrics of the headline model (Appendices I, VI, VII; Figs. of Results).
docs/results/ablation_sources_contiguous.json	Representation ablation (Table 5).
docs/results/ ablation_sources_RANDOM_BLOCK_FOLDS_ superseded.json	The same ablation with random blocks (Appendix II).
docs/results/audit_full_dataset.json	Shortcut audit (Table 3).
docs/results/leakage_analysis.json	Scheme comparison, purge and gap analyses (Tables 6, 7, 8).
docs/results/dataset_stats.json	Dataset and segmentation statistics (Table 1).
docs/results/session_structure.json	Adjacent-versus-random similarity analysis.
docs/results/hds_benchmark.json	Denoising benchmark.
docs/results/feature_stats.json	Per-class feature statistics.
docs/results/curve_summary.json, cells_summary.json	ROC and precision-recall areas; accuracy by cell count.
scripts/analysis_*.py, scripts/benchmark_hds.py, scripts/make_figures.py	Scripts that produce the above files and the figures.
paper/build_paper.py	Script that assembles this manuscript from the result files.